

MC23 – Algoritmos

Resumo detalhado para a AV1
(IMPA Tech, 2026.2 – Prof. Uéverton Souza)

Baseado nos slides, aulas manuscritas, monitorias, listas, Testes 1 e 2;
complementado com Szwarcfiter & Markenzon (*Estruturas de Dados e seus Algoritmos*)
e Cormen et al. (*Introduction to Algorithms*, 3. ed.)

AV1: terça-feira, 22 de setembro de 2026

Como usar este resumo

- Ele foi escrito para quem **já estudou a matéria** e quer se localizar rápido: cada seção começa com o essencial, depois vêm as demonstrações e os detalhes que costumam ser cobrados.
- **Caixas verdes**: como o tema cai na AV1 (o formato da prova foi anunciado em aula — ver abaixo).
- **Caixas vermelhas**: armadilhas e erros frequentes (inclusive erros encontrados em slides/-monitorias).
- **Caixas azuis**: resumo do que precisa estar na ponta da língua.
- **Caixas laranjas**: complementos (livros/pesquisas) que não foram o foco das aulas.
- Convenção de heap do curso (Szwarcfiter): **heap de máximo, vetor indexado a partir de 1**, pai de i é $\lfloor i/2 \rfloor$, filhos $2i$ e $2i + 1$. Balanço AVL: $\text{bal}(v) = h_D(v) - h_E(v)$.
- Todos os exemplos numéricos (AVL, heaps, radix, subvetor máximo, inversões) foram *conferidos por simulação*.

Formato da AV1 (anunciado na aula de revisão de 18/09)

Questão	O que será pedido	Onde estudar
Q1	Árvore AVL : saber como a estrutura funciona, resolver uma tarefa nela, manipular (inserir/remover, rotações, balanços).	Seção 5
Q2	Union-Find (a mais fácil): diferentes formas de implementar e em que situações cada uma é melhor.	Seção 4 (Tabela 1)
Q3	Heap : saber <i>demonstrar</i> propriedades de heap.	§2.7 (provas); depois §2, §3
Q4	Dado um código genérico: qual estrutura de dados usar e qual ordenação usar.	Seções 3.4, 6.4, 6.5
Q5	Divisão e conquista : dado um problema, escrever o pseudocódigo.	Seção 8

Sumário

Mapa: aulas e materiais → seções

4

Cola: complexidades de tudo	5
1 Complexidade de algoritmos	6
1.1 Medidas de complexidade	6
1.2 Notações assintóticas	6
1.3 Complexidade de problemas $\times$ de algoritmos	6
1.4 Recorrências	7
1.5 Complexidade amortizada	7
2 Listas de prioridades e heaps binários	8
2.1 Lista de prioridades	8
2.2 Definição e representação	8
2.3 Subir e descer	9
2.4 Inserção, remoção, alteração	9
2.5 Construção em tempo linear	10
2.6 Heapsort	11
2.7 Propriedades de heap e suas demonstrações	11
3 Heaps avançados: binomiais e de Fibonacci	12
3.1 Árvores binomiais	13
3.2 Heap binomial	13
3.2.1 Operações	13
3.3 Heap de Fibonacci	14
3.3.1 Por que funciona: grau máximo $O(\log n)$	15
3.3.2 Análise amortizada	15
3.4 Comparação e escolha da estrutura (Q4 e Teste 1)	16
4 Conjuntos disjuntos: Union-Find	17
4.1 O problema	17
4.2 Implementações	17
4.3 Rank + compressão: $O(m \log^* n)$ (demonstração da aula de 25/08)	19
4.4 Qual implementação usar? (Q2 da AV1)	19
4.5 Exercícios resolvidos (monitoria de revisão e lista)	21
5 Árvores binárias de busca e árvores AVL	21
5.1 Árvore binária de busca (ABB)	21
5.2 Árvores AVL (Adel'son-Vel'skii e Landis, 1962)	22
5.3 Inserção: atualização dos balanços	22
5.4 As rotações	23
5.5 Pseudocódigo (slides do professor)	23
5.6 Remoção em AVL	25
5.7 Exemplos resolvidos (conferidos por simulação)	25
5.8 Exercícios da lista (Union-Find e AVL)	27
5.9 Árvores rubro-negras (pesquisa)	28
6 Ordenação	29
6.1 Ordenação por comparação	29
6.2 Limite inferior $\Omega(n \log n)$ para ordenação por comparação	30
6.3 Ordenação em tempo linear	30
6.4 Qual algoritmo de ordenação usar?	32
6.5 Q4: dado um código, qual estrutura e qual ordenação?	32

7	Seleção: o algoritmo BFPRT (mediana das medianas)	33
7.1	O algoritmo	33
7.2	Análise	34
8	Divisão e conquista	34
8.1	Contagem de inversões (aula de 11/09)	35
8.2	Subvetor contíguo de soma máxima (Teste 2, Exercício 1)	35
8.3	Multiplicação de matrizes: Strassen (Teste 2, Exercício 2)	36
8.4	Outros exemplos clássicos de D&C (treino para a Q5)	37
9	Complementos	38
9.1	Tabelas hash (pesquisa)	38
9.2	Programação dinâmica	39
10	Banco de questões com respostas	39
10.1	Testes aplicados	39
10.2	Lista – heaps binários, binomiais e de Fibonacci	39
10.3	Lista – Union-Find e AVL	41
10.4	Monitoria de revisão: respostas curtas	41

Mapa: aulas e materiais → seções

Aula	Data	Conteúdo	Seção
1	11/08	Complexidade, notações assintóticas, limites superior/inferior, polinomial $\times$ exponencial	1
2	12/08	Listas de prioridades e heaps (binário), heapsort	2
3	18/08	Heaps binários (resumo de custos) e heaps binomiais	2, 3
4	21/08	Heaps de Fibonacci e complexidade amortizada	1.5, 3
5	25/08	Union-Find: listas, florestas, rank, compressão, análise $O(m \log^* n)$	4
6	28/08	Árvores binárias de busca e AVL	5
7	01/09	Ordenação por comparação (heap/merge/quick), limite inferior $\Omega(n \log n)$	6.1
8	04/09	Ordenação linear: counting, radix, bucket (e American Flag)	6.3
9	08/09	Seleção: algoritmo BFPRT (mediana das medianas)	7
10	11/09	Divisão e conquista; contagem de inversões	8
11	15/09	Teste 2 (ordenação e D&C)	10.1
12–13	16 e 18/09	Exercícios; revisão (AVL, D&C); formato da AV1	—
Monitorias		Heaps binomiais/Fibonacci (26/08), Union-Find, AVL (aula prática 4), Revisão Heap/UF/AVL	3, 4, 5, 10
Pesquisas	até 19/09	Árvores rubro-negras; Tabelas hash	5.9, 9.1
Listas		Heaps (binário/binomial/Fibonacci); Union-Find e AVL	10

Obs.: os slides de Programação Dinâmica, Gulosos, Matroides, AGM e Grafos pertencem à parte da AV2 no andamento real do curso; só PD aparece aqui, como complemento curto (§9.2).

Cola: complexidades de tudo

Listas de prioridades (n elementos; * = amortizado; heap de máximo)

Operação	Lista não ord.	Lista ordenada	Heap binário	Heap binomial	Heap de Fibonacci
Seleção (máx.)	$\Theta(n)$	$\Theta(1)$	$O(1)$	$O(\log n)$ ou $O(1)$ c/ ponteiro	$O(1)$
Inserção	$\Theta(1)$	$\Theta(n)$	$O(\log n)$	$O(\log n)$; $O(1)$ esperado/*	$O(1)^*$
Remoção do máx.	$\Theta(n)$	$\Theta(1)$	$O(\log n)$	$O(\log n)$	$O(\log n)^*$
Aumentar prioridade	$\Theta(n)$	$\Theta(n)$	$O(\log n)$	$O(\log n)$	$O(1)^*$
Diminuir prioridade	$\Theta(n)$	$\Theta(n)$	$O(\log n)$	$O(\log n)$	$O(\log n)^*$
Construção	$\Theta(n)$	$\Theta(n \log n)$	$O(n)$	$O(n)$	$O(n)$
União (fusão)	$O(1)$ encad.	$O(n)$	$O(n)$	$O(\log n)$	$O(1)^*$

Union-Find (n elementos, m operações)

Implementação	MAKE-SET	FIND	UNION
Listas encadeadas c/ ponteiro ao representante (união pelo tamanho)	$O(1)$	$O(1)$	$O(\log n)^*$
Floresta ingênua	$O(1)$	$O(n)$	$O(1)$ (dadas as raízes)
Floresta + união por tamanho/rank	$O(1)$	$O(\log n)$	$O(1)$ (dadas as raízes)
Floresta + rank + compressão de caminhos	$O(1)$	$O(m \log^* n)$ no total (justo: $O(m \alpha(m, n))$)	

Árvores de busca ABB: busca/inserção/remoção $\Theta(h)$, com h entre $\lfloor \log n \rfloor + 1$ e n . AVL: tudo $O(\log n)$ no pior caso; inserção faz ≤ 1 rotação (simples ou dupla); remoção pode fazer $O(\log n)$ rotações.

Ordenação

Algoritmo	Pior caso	Caso médio	Melhor caso	Memória extra	Estável?	In-place?
Inserção / Bolha	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n)$	$O(1)$	sim	sim
Mergesort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$	sim	não
Heapsort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(1)$	não	sim
Quicksort	$\Theta(n^2)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(\log n)$ pilha (média)	não	sim
Counting sort	$\Theta(n + k)$	$\Theta(n + k)$	$\Theta(n + k)$	$\Theta(n + k)$	sim	não
Radix sort (LSD)	$\Theta(d(n + b))$	idem	idem	$\Theta(n + b)$	sim	não
Bucket sort	$\Theta(n \log n)$ ou $\Theta(n^2)$	$\Theta(n)$ (unif.)	$\Theta(n)$	$\Theta(n)$	depende	não

Outros Limite inferior p/ ordenação por comparação: $\Omega(n \log n)$. Seleção (k -ésimo menor): BFPRT $O(n)$ pior caso; quickselect $O(n)$ esperado, $O(n^2)$ pior. Contagem de inversões e subvetor de soma máxima (D&C): $O(n \log n)$. Strassen: $O(n^{\log_2 7}) \approx O(n^{2,807})$.

1 Complexidade de algoritmos

1.1 Medidas de complexidade

Seja A um algoritmo e $\{E_1, \dots, E_m\}$ todas as entradas possíveis de tamanho n ; T_i é o número de passos de A sobre E_i .

- **Pior caso:** $\max_i T_i$ (é a medida padrão do curso).
- **Caso médio:** $\sum_i p_i T_i$, onde p_i é a probabilidade de E_i .
- **Melhor caso:** $\min_i T_i$.
- **Amortizada:** custo de uma *sequência* de operações dividido pelo número de operações, no pior caso da sequência (§1.5). Não é média probabilística!

Tamanho da entrada. É o comprimento da codificação da entrada (em bits/símbolos). Isso importa com números: um algoritmo é **pseudopolinomial** se é polinomial quando os números da entrada são codificados em *unário*. Exemplo do curso: o *counting sort* é $O(n + k)$; se $k = \Theta(2^n)$ ele é exponencial no tamanho da entrada (um número k ocupa só $\log k$ bits).

1.2 Notações assintóticas

Sejam f, g funções (assintoticamente) não negativas.

Notação	Definição	Via limite (se existir)	Leitura
$f = O(g)$	$\exists c > 0, n_0 : f(n) \leq c g(n) \forall n \geq n_0$	$\lim f/g < \infty$	“ $\leq$ ”
$f = \Omega(g)$	$\exists c > 0, n_0 : f(n) \geq c g(n) \forall n \geq n_0$	$\lim f/g > 0$	“ $\geq$ ”
$f = \Theta(g)$	$f = O(g)$ e $f = \Omega(g)$	$\lim f/g = c \in (0, \infty)$	“ $=$ ”
$f = o(g)$	$\forall c > 0 \exists n_0 : f(n) < c g(n)$	$\lim f/g = 0$	“ $<$ ” (não justo)
$f = \omega(g)$	$\forall c > 0 \exists n_0 : f(n) > c g(n)$	$\lim f/g = \infty$	“ $>$ ” (não justo)

Exemplo dos slides: $2n^2 + 1$ é $O(n^3)$, $O(n^2)$, não é $O(n)$; é $\Omega(n)$, $\Omega(n^2)$, não é $\Omega(n^3)$; é $\Theta(n^2)$. $O(g)$ também denota o *conjunto* das funções f com $f = O(g)$ (escreve-se $f \in O(g)$).

Propriedades úteis. Transitividade; $f = O(g) \iff g = \Omega(f)$; $O(f) + O(g) = O(\max\{f, g\})$; $\log_a n = \Theta(\log_b n)$ (a base do log não importa); $\log(n!) = \Theta(n \log n)$; $(\log n)^c = o(n^\epsilon)$ e $n^c = o(a^n)$ para $a > 1$.

$$1 < \log \log n < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < n^{\log n} < 2^n < n! < n^n.$$

1.3 Complexidade de problemas $\times$ de algoritmos

Definição 1.1.

- Um **limite superior** para o problema Π é uma função $f(n)$ tal que *existe* algum algoritmo que resolve Π com no máximo $f(n)$ passos no pior caso. (A complexidade de pior caso de qualquer algoritmo para Π é um limite superior para Π .)
- Um **limite inferior** para Π é uma função $f(n)$ tal que *todo* algoritmo para Π precisa de pelo menos $f(n)$ passos no pior caso.
- Um algoritmo é **ótimo** se sua complexidade de pior caso é igual a um limite inferior do problema (ex.: mergesort e heapsort para ordenação por comparação).

As notações são só sobre funções: nada impede dizer “o pior caso de A é $\Omega(n^3)$ ” ou “o limite inferior de Π é $O(\log n)$ ”.

Polinomial $\times$ exponencial. Consenso: um algoritmo é útil na prática se é **polinomial** ($O(n^c)$; classe P = problemas com algoritmo polinomial). Exponenciais: $2^{O(n)}$, k^n , $n!$, 2^{n^2} , $n^{\log n}$. Polinomiais aproveitam melhor o avanço tecnológico: com máquina $10\times$ mais rápida, o maior n resolvido em um dia é *multiplicado* por uma constante (n^2 : $\times 3,16$), enquanto para 2^n só *soma* uma constante ($40 \rightarrow 43$).

1.4 Recorrências

Algoritmos recursivos (D&C) têm tempo dado por recorrências $T(n) = aT(n/b) + f(n)$: a subproblemas de tamanho n/b e custo $f(n)$ para dividir e combinar.

Árvore de recursão. Some o custo por nível. Ex.: $T(n) = 2T(n/2) + cn$: há $\log_2 n$ níveis, cada um custando $cn \Rightarrow T(n) = \Theta(n \log n)$ (mergesort, contagem de inversões, subvetor máximo).

Substituição (indução). Chuta-se $T(n) \leq cg(n)$ e prova-se por indução forte (é assim que o slide prova $T(n) \leq 20cn$ na BFPRT, §7).

Teorema 1.2 (Teorema Mestre – Cormen §4.5). *Seja $T(n) = aT(n/b) + f(n)$ com $a \geq 1$, $b > 1$, e seja $E = \log_b a$ (o “expoente crítico”).*

1. Se $f(n) = O(n^{E-\epsilon})$ para algum $\epsilon > 0$: $T(n) = \Theta(n^E)$ (as folhas dominam).
2. Se $f(n) = \Theta(n^E)$: $T(n) = \Theta(n^E \log n)$ (todos os níveis pesam igual).
3. Se $f(n) = \Omega(n^{E+\epsilon})$ e $af(n/b) \leq cf(n)$ para algum $c < 1$: $T(n) = \Theta(f(n))$ (a raiz domina).

Recorrência	Exemplo	Solução
$T(n) = T(n/2) + O(1)$	busca binária	$\Theta(\log n)$
$T(n) = 2T(n/2) + O(1)$	máximo por D&C	$\Theta(n)$
$T(n) = 2T(n/2) + \Theta(n)$	mergesort, inversões, subvetor máximo	$\Theta(n \log n)$
$T(n) = 8T(n/2) + \Theta(n^2)$	produto de matrizes por blocos	$\Theta(n^3)$
$T(n) = 7T(n/2) + \Theta(n^2)$	Strassen	$\Theta(n^{\log_2 7}) \approx \Theta(n^{2,807})$
$T(n) = T(n/5) + T(3n/4) + \Theta(n)$	BFPRT	$\Theta(n)$ (pois $\frac{1}{5} + \frac{3}{4} < 1$)
$T(n) = T(n-1) + \Theta(n)$	quicksort no pior caso	$\Theta(n^2)$
$T(n) = T(9n/10) + T(n/10) + \Theta(n)$	quicksort “desbalanceado fixo”	$\Theta(n \log n)$

Teorema 1.3 (D&C c -balanceado – slide de D&C). *Se um algoritmo divide a instância em k partes de tamanho no máximo n/c ($c > 1$), com partes de tamanhos que diferem por constante, e gasta tempo linear para dividir e combinar, então roda em $O(n \log n)$.*

Ideia. Na árvore de recursão, os subproblemas de um mesmo nível são disjuntos, então cada nível custa $O(n)$; como o tamanho cai por um fator c a cada nível, há $O(\log_c n)$ níveis. $\square$

Somatórios que aparecem. $\sum_{i=1}^n i = \frac{n(n+1)}{2}$; $\sum_{k \geq 0} x^k = \frac{1}{1-x}$ ($|x| < 1$); $\sum_{k \geq 0} \frac{k}{2^k} = 2$; $\sum_{k=0}^{\log n} \frac{n}{2^k} \leq 2n$; $\sum_{i=1}^n \frac{1}{i} = \Theta(\log n)$; $\log(n!) = \sum \log i = \Theta(n \log n)$.

1.5 Complexidade amortizada

A análise amortizada considera uma **sequência** de operações: cada operação individual pode ser cara, mas o custo total da sequência (no pior caso) é pequeno. **Não envolve probabilidade.**

Visão do banqueiro (créditos). Cada operação recebe um número fixo de créditos (seu custo amortizado). Operações baratas “poupam” créditos, que pagam operações caras depois. Se o saldo nunca fica negativo, o custo real total $\leq$ soma dos custos amortizados.

Visão do físico (potencial). Escolhe-se $\Phi(E) \geq 0$ para cada configuração E . Custo amortizado: $a_i = t_i + \Phi(E_i) - \Phi(E_{i-1})$. Somando (telescópica):

$$\sum_{i=1}^m t_i = \sum_{i=1}^m a_i - \Phi_m + \Phi_0 \leq \sum_{i=1}^m a_i \quad \text{se } \Phi_m \geq \Phi_0.$$

Exemplo 1.4 (Pilha com multi-desempilhamento – Szwarcfiter §8.2). Cada operação faz $k \geq 0$ *retira* e depois um *insere*; no pior caso uma operação custa $\Theta(n)$. Com Φ = número de elementos na pilha: $a_i = (1 + k) + (p - k + 1) - p = 2$. Logo m operações custam $\leq 2m = O(m)$.

Exemplo 1.5 (Contador binário). Incrementar um contador de k bits pode trocar k bits, mas m incrementos a partir de 0 trocam no total $\leq 2m$ bits: com Φ = número de bits 1, um incremento que zera t bits e liga um tem $a = (t + 1) + (1 - t) = 2$. É exatamente o argumento da **inserção em heap binomial** (§3.1): inserir = somar 1 em binário.

Pontos-chave

“Amortizado” $\neq$ “caso médio”. Amortizado é *garantia de pior caso para a sequência*; uma operação isolada ainda pode ser cara. Isso é decisivo na escolha entre heap binomial e de Fibonacci quando o sistema exige **previsibilidade de cada operação individual** (Teste 1, Q1).

2 Listas de prioridades e heaps binários

2.1 Lista de prioridades

Tabela em que cada dado tem uma **prioridade**; operações: (1) *seleção* do de maior prioridade; (2) *inserção*; (3) *remoção* do de maior prioridade; (4) *alteração* de prioridade; (5) *construção* a partir de um conjunto inicial. (Fusão entra com os heaps avançados.)

Operação	Lista não ordenada	Lista ordenada	Heap
seleção	$\Theta(n)$	$\Theta(1)$	$O(1)$
inserção	$\Theta(1)$	$\Theta(n)$	$O(\log n)$
remoção	$\Theta(n)$	$\Theta(1)$	$O(\log n)$
alteração	$\Theta(n)$	$\Theta(n)$	$O(\log n)$
construção	$\Theta(n)$	$\Theta(n \log n)$	$O(n)$

As duas listas têm alguma operação de atualização $\Omega(n)$; o heap equilibra tudo em $O(\log n)$.

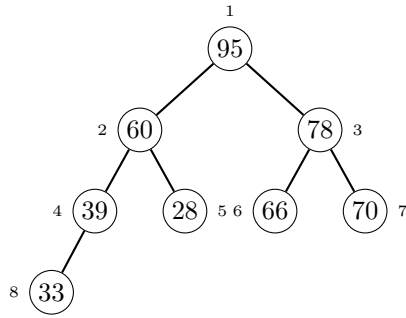
2.2 Definição e representação

Definição 2.1 (Heap – convenção do curso). Um **heap** (de máximo) é uma lista linear $s_1, \dots, s_n$ tal que

$$s_i \leq s_{\lfloor i/2 \rfloor} \quad \text{para todo } 1 < i \leq n.$$

(Heap de *mínimo*: $s_i \geq s_{\lfloor i/2 \rfloor}$.)

Visualiza-se como uma **árvore binária completa** numerada por níveis, da esquerda para a direita: o nó i tem pai $\lfloor i/2 \rfloor$, filho esquerdo $2i$ e direito $2i + 1$ (se $\leq n$). A condição diz: *cada nó tem prioridade $\geq$ à dos filhos*. Não há ponteiros: é só um vetor.



1	2	3	4	5	6	7	8
95	60	78	39	28	66	70	33

Fatos de índice (usados em todas as provas):

- o nó i está no nível $\lfloor \log_2 i \rfloor + 1$ (raiz no nível 1); a árvore tem $\lfloor \log_2 n \rfloor + 1$ níveis;
- as **folhas** são exatamente as posições $\lfloor n/2 \rfloor + 1, \dots, n$ (pois i tem filho sse $2i \leq n$); há $\lceil n/2 \rceil$ folhas;
- convenção de altura do Szwarcfiter/slides: **folhas têm altura 1**.

2.3 Subir e descer

subir(i) – após *aumentar* a prioridade de i

Procedimento SUBIR(i)

$j := \lfloor i/2 \rfloor$

se $j \geq 1$ **então**

se $T[i].chave > T[j].chave$ **então**

$T[i] \leftrightarrow T[j]$

SUBIR(j)

descer(i, n) – após *diminuir* a prioridade de i

Procedimento DESCER(i, n)

$j := 2i$

se $j \leq n$ **então**

se $j < n$ e $T[j+1].chave > T[j].chave$

então

$j := j + 1$

% maior filho

se $T[i].chave < T[j].chave$ **então**

$T[i] \leftrightarrow T[j]$

DESCER(j, n)

Ambos percorrem um caminho da árvore: $O(\log n)$. **Descer escolhe o maior filho** (se trocasse com o menor, o maior filho ficaria abaixo de um valor menor).

2.4 Inserção, remoção, alteração

Inserção, remoção do máximo e alteração de prioridade

Procedimento INSERIR($novo$)

% M = capacidade

se $n < M$ **então**

$T[n+1] := novo; n := n + 1; \text{SUBIR}(n)$

senão overflow

Procedimento REMOVER

se $n \neq 0$ **então**

$\text{agir}(T[1]); T[1] := T[n]; n := n - 1; \text{DESCER}(1, n)$

senão underflow

Procedimento ALTERAR(i, p)

$antiga := T[i].chave; T[i].chave := p$

se $p > antiga$ **então**

SUBIR(i)

% aumentou $\Rightarrow$ sobe

senão

DESCER(i, n)

% diminuiu $\Rightarrow$ desce

Erro clássico (monitoria de revisão, Q5)

Um aluno escreveu: “ $H[i] := p$; **se** $H[i] > H[\lfloor i/2 \rfloor]$ **então** Descer(i) **senão** Subir(i)”. Está **invertido**: se o novo valor ficou maior que o pai, ele viola a propriedade *com o pai* e precisa **subir**; se não ficou maior que o pai, a única violação possível é com os filhos, e ele precisa **descer**.
 Versão correta: **se** $H[i] > H[\lfloor i/2 \rfloor]$ (com $i > 1$) **então** Subir(i) **senão** Descer(i, n).

Exemplo 2.2 (Traços conferidos – heap $H = [95, 60, 78, 39, 28, 66, 70, 33]$).

- **Aumentar** o nó 6 de 66 para 98: sobe trocando com 78 (pos. 3) e com 95 (pos. 1): $[98, 60, 95, 39, 28, 78, 70, 33]$.
- **Remover a raiz**: coloca-se o último (33) na raiz, $[33, 60, 78, 39, 28, 66, 70]$; desce trocando com 78 (maior filho) e depois com 70: $[78, 60, 70, 39, 28, 66, 33]$.
- **Diminuir** a raiz de 95 para 37: desce trocando com 78 e com 70: $[78, 60, 70, 39, 28, 66, 37, 33]$.
- **Inserir** 73: entra na pos. 9 (filho de 39), sobe trocando com 39 e com 60: $[95, 73, 78, 60, 28, 66, 70, 33, 39]$.

2.5 Construção em tempo linear

1ª tentativa (inserções sucessivas): **para** $i := 2, \dots, n$ **faça** subir(i) — custa $O(n \log n)$ (e $\Theta(n \log n)$ no pior caso, p.ex. entrada crescente).

2ª tentativa (*arranjar*): as folhas ($i > \lfloor n/2 \rfloor$) já são heaps; corrige-se de baixo para cima.

arranjar(n)

Procedimento ARRANJAR(n)

para $i := \lfloor n/2 \rfloor$ **decr. até** 1 **faça**
 DESCER(i, n)

Correção: invariante “antes de processar i , as subárvores de raízes $i+1, \dots, n$ são heaps”. Ao chamar $\text{descer}(i, n)$, as duas subárvores dos filhos de i já são heaps, e descer transforma a subárvore de i em heap.

Lema 2.3 (Lista de heaps, ex. 6). *Numa árvore binária completa com n nós (folhas com altura 1), no máximo $\lceil n/2^j \rceil$ nós têm altura j .*

Demonstração. Numa árvore completa o caminho mais longo a partir de i é o da esquerda: $i \rightarrow 2i \rightarrow 4i \rightarrow \dots$. Logo o nó i tem altura $\geq j$ sse seu descendente $j-1$ níveis abaixo existe, isto é, sse $i2^{j-1} \leq n$, ou seja, $i \leq \lfloor n/2^{j-1} \rfloor$. Assim, com $x = n/2^{j-1}$ e $m = \lfloor x \rfloor$,

$$\#\{\text{altura} = j\} = \lfloor x \rfloor - \lfloor x/2 \rfloor = m - \lfloor m/2 \rfloor = \lceil m/2 \rceil \leq \lceil x/2 \rceil = \lceil n/2^j \rceil. \quad \square$$

Teorema 2.4 (Lema 6.1 do Szwarcfiter / slide). *arranjar(n) roda em $O(n)$.*

Demonstração. $\text{descer}(i, n)$ num nó de altura j faz no máximo $j-1$ trocas. Pelo Lema 2.3, o total é

$$\sum_{j=2}^h (j-1) \left\lceil \frac{n}{2^j} \right\rceil \leq \sum_{j \geq 2} (j-1) \left(\frac{n}{2^j} + 1 \right) = n \sum_{j \geq 2} \frac{j-1}{2^j} + O(\log^2 n) = n \cdot 1 + O(\log^2 n) = O(n),$$

pois $\sum_{j \geq 1} \frac{j-1}{2^j} = \sum_{j \geq 1} \frac{j}{2^j} - \sum_{j \geq 1} \frac{1}{2^j} = 2 - 1 = 1$. (O slide faz a mesma conta separando j par e ímpar e usando $\frac{j}{2^{j+1}} \leq \frac{j-1}{2^j}$.) $\square$

Intuição: metade dos nós (folhas) não custa nada, $1/4$ custa ≤ 1 , $1/8$ custa $\leq 2, \dots$; só a raiz custa $\log n$.

Exemplo 2.5 (Traços do *arranjar* (conferidos)).

- Lista de heaps, ex. 5: [18, 25, 41, 34, 14, 10, 52, 50, 48] ($n = 9$, processa $i = 4, 3, 2, 1$).
 $i = 4$: $34 \leftrightarrow 50$; $i = 3$: $41 \leftrightarrow 52$; $i = 2$: $25 \leftrightarrow 50$, depois $25 \leftrightarrow 48$; $i = 1$: $18 \leftrightarrow 52$, depois $18 \leftrightarrow 41$.
 Resultado: [52, 50, 41, 48, 14, 10, 18, 34, 25].
- Monitoria (Q4): [20, 15, 30, 40, 10, 25, 5] processa $i = 3, 2, 1$: $i = 3$ nada; $i = 2$: $15 \leftrightarrow 40$; $i = 1$: $20 \leftrightarrow 40$. Resultado [40, 20, 30, 15, 10, 25, 5].
- Monitoria (Q13): [7, 2, 15, 9, 1, 20, 13]: $i = 3$: $15 \leftrightarrow 20$; $i = 2$: $2 \leftrightarrow 9$; $i = 1$: $7 \leftrightarrow 20$, $7 \leftrightarrow 15$. Resultado [20, 9, 15, 2, 1, 7, 13] (o mesmo que por inserções sucessivas neste caso).

2.6 Heapsort

Heapsort (Szwarcfiter, Alg. 7.6)

```

Procedimento HEAPSORT( $T, n$ )
  ARRANJAR( $n$ )                                     %  $O(n)$ 
  para  $m := n$  decrecendo até 2 faça
     $T[1] \leftrightarrow T[m]$                              % o máximo vai para o fim
    DESCER( $1, m - 1$ )                                %  $O(\log m)$ 

```

Complexidade $O(n) + \sum_m O(\log m) = \Theta(n \log n)$ em *todos* os casos (mesmo com a entrada ordenada). **In-place** (só troca no vetor) e **não estável** (a troca da raiz com o último embaralha iguais). É ótimo entre os algoritmos por comparação. Ex.: [18, 25, 41, 34, 14, 10, 52, 50, 48] $\rightarrow$ [10, 14, 18, 25, 34, 41, 48, 50, 52].

2.7 Propriedades de heap e suas demonstrações

Q3 da AV1: “saber demonstrar propriedades de heap”

Quase toda prova de heap usa só três ferramentas: (i) a definição $s_i \leq s_{\lfloor i/2 \rfloor}$ + **transitividade ao longo de um caminho**; (ii) os **fatos de índice** (pai, filhos, folhas, níveis); (iii) **indução** (na altura, no nível, ou no número de nós). Para “provar ou dar contraexemplo”, teste primeiro exemplos pequenos ($n \leq 8$).

Proposição 2.6 (Ancestral domina). *Se j é ancestral de i num heap, então $s_i \leq s_j$. Em particular: todo caminho da raiz a uma folha é não crescente, a raiz é o máximo e cada subárvore é um heap.*

Demonstração. O caminho de i até j é $i, \lfloor i/2 \rfloor, \lfloor i/4 \rfloor, \dots, j$; aplicando a definição em cada aresta e a transitividade de $\leq$, $s_i \leq s_{\lfloor i/2 \rfloor} \leq \dots \leq s_j$. A raiz é ancestral de todos. A condição do heap numa subárvore é a mesma condição restrita a ela. $\square$

Proposição 2.7 (O mínimo está numa folha). *Num heap de máximo com chaves distintas, o menor elemento está numa folha, isto é, numa posição $> \lfloor n/2 \rfloor$.*

Demonstração. Se i não é folha, tem filho $2i$ e $s_{2i} < s_i$ (distintas); logo s_i não é o mínimo. (Com repetições: algum mínimo está numa folha.) $\square$

Proposição 2.8 (Onde está o k -ésimo maior). *Com chaves distintas, o k -ésimo maior elemento está num nível $\leq k$, isto é, numa posição $\leq 2^k - 1$. Em particular o 2^o maior é filho da raiz (posição 2 ou 3), e o 3^o maior está nas posições 2, $\dots$, 7.*

Demonstração. Pela Prop. 2.6, todos os ancestrais de x são maiores que x . Se x é o k -ésimo maior, só existem $k - 1$ elementos maiores, então x tem no máximo $k - 1$ ancestrais, ou seja, está no nível $\leq k$. As posições dos níveis 1.. k são $1, \dots, 2^k - 1$. $\square$

Proposição 2.9 (Altura). *Um heap com n elementos tem altura $\lfloor \log_2 n \rfloor + 1$ (em níveis) $= \Theta(\log n)$; por isso subir/descer/inserir/remover são $O(\log n)$.*

Demonstração. O último nó, n , está no nível $\lfloor \log_2 n \rfloor + 1$, e numa árvore completa todos os níveis anteriores estão cheios: $2^{h-1} \leq n \leq 2^h - 1$, logo $h = \lfloor \log_2 n \rfloor + 1$. $\square$

Proposição 2.10 (Relação com ordenação). *Toda lista ordenada de forma não crescente é um heap; a recíproca é falsa (ex.: $[3, 1, 2]$). Por isso construir um heap é mais fácil que ordenar ($O(n)$ vs. $\Omega(n \log n)$) — e, como heapsort ordena usando construção + n remoções, não existe heap com construção e remoção ambas $o(\log n)$ amortizado por comparação (senão ordenaríamos em $o(n \log n)$).*

Proposição 2.11 (Troca de dois elementos – Lista de heaps, ex. 2 e 3). *Seja S um heap e $i < j$.*

(a) *Se $s_i < s_j$, trocar s_i com s_j **não** produz necessariamente um heap.*

(b) *Se $s_i > s_j$, trocar s_i com s_j **não** produz necessariamente um heap.*

Contraexemplos (verificados). (a) $S = [10, 9, 1, 8, 7, 0, 0, 6]$ é heap; $i = 3$ (1), $j = 4$ (8). Após a troca, $[10, 9, 8, 1, 7, 0, 0, 6]$: a posição 8 (6) tem pai na posição 4 (1) e $6 > 1$. Intuição: o valor pequeno vai para a posição j , cujos filhos podem ser maiores que ele. (b) $S = [10, 9]$, $i = 1$, $j = 2$: após a troca $[9, 10]$, filho maior que o pai. Intuição: o valor pequeno sobe para i e pode ficar abaixo de um descendente maior. $\square$

Proposição 2.12 (Número de heaps – Lista de heaps, ex. 4). *Seja $H(n)$ o número de permutações de $1, \dots, n$ que são heaps (vale o mesmo para min-heap). Então $H(0) = H(1) = 1$ e*

$$H(n) = \binom{n-1}{L} H(L) H(n-1-L),$$

onde L é o número de nós da subárvore esquerda da raiz: com $h = \lfloor \log_2 n \rfloor$ e $u = n - (2^h - 1)$ nós no último nível, $L = (2^{h-1} - 1) + \min\{u, 2^{h-1}\}$. Valores: 1, 1, 2, 3, 8, 20, 80, 210, 896, 3360 para $n = 1..10$ (conferido por força bruta até $n = 8$).

Demonstração. A raiz é necessariamente n (máximo). O formato da árvore é fixo, então L é determinado por n . Escolhem-se quais L dos $n - 1$ valores restantes vão para a esquerda ($\binom{n-1}{L}$ modos); cada lado precisa ser um heap sobre seus valores, e o número de heaps só depende da quantidade de valores ($H(L)$ e $H(n - 1 - L)$). $\square$

Proposição 2.13 (Correção de subir e descer). (a) *Se T era heap e a chave de i aumentou, após subir(i) T é heap.* (b) *Se as subárvores dos filhos de i são heaps (em particular se T era heap e a chave de i diminuiu), após descer(i, n) a subárvore de i é heap.*

Esboço. (a) Invariante: a única violação possível é entre o nó corrente e seu pai. Ao trocar com o pai j (quando $s_i > s_j$), o antigo s_j desce para i e é $\geq$ aos filhos de i (eram seus descendentes); o novo valor em j é maior que o antigo, então não viola com os filhos de j . Repete-se para cima. (b) Invariante: a única violação é entre o nó corrente e seus filhos. Troca-se com o *maior* filho: ele passa a dominar o outro filho e o valor que desceu. A violação desce um nível; termina numa folha ou quando não há violação. $\square$

Pontos-chave

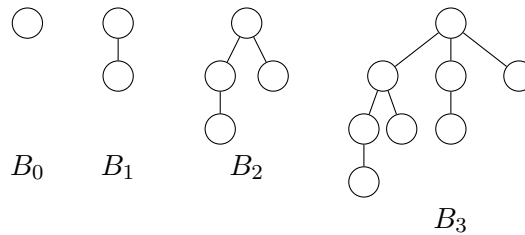
Heap = vetor + “pai $\geq$ filho”. Subir após aumento, descer após diminuição (maior filho!). Inserir = pôr no fim + subir; remover = último na raiz + descer. Construção linear pelo *arranjar* (de $\lfloor n/2 \rfloor$ até 1) graças a $\leq \lceil n/2^j \rceil$ nós de altura j . Heapsort $\Theta(n \log n)$, in-place, não estável.

3 Heaps avançados: binomiais e de Fibonacci

Motivação. No heap binário (vetor) a **fusão** (união de duas filas) custa $O(n)$: concatena-se os vetores e reconstrói-se com *arranjar*. Em aplicações com muitas fusões (setores de controle, Dijkstra/Prim em grafos grandes) isso é inviável. O heap binomial faz fusão em $O(\log n)$ pior caso; o de Fibonacci, em $O(1)$ amortizado.

3.1 Árvores binomiais

Definição 3.1. B_0 é um único nó. Para $k \geq 1$, B_k é obtida **subordinando** (*link*) uma B_{k-1} à outra: a raiz de uma vira filho (mais à direita/novo primeiro filho) da raiz da outra. Num heap, a raiz de *maior* prioridade fica por cima.



Proposição 3.2 (Propriedades de B_k – provas por indução em k).

1. B_k tem exatamente 2^k nós ($2 \cdot 2^{k-1}$).
2. O grau da raiz (a **ordem** de B_k) é k , e é o grau máximo da árvore (o link dá mais um filho à raiz).
3. A altura de B_k é k (medida em arestas) (a subordinada fica um nível abaixo: $\max\{k-1, 1+(k-1)\}$).
4. Há exatamente $\binom{k}{j}$ nós na profundidade j ($\binom{k-1}{j} + \binom{k-1}{j-1} = \binom{k}{j}$; daí o nome).
5. Os filhos da raiz de B_k são raízes de $B_{k-1}, B_{k-2}, \dots, B_1, B_0$.

Convenção de altura: aula $\times$ livro

Na aula (18/08) e na monitoria “Heaps Binomiais”: B_k **tem altura** k (conta arestas). No Szwarcfiter e na monitoria “Heap-fibonacci-binomial”: B_h **tem altura** $h+1$ (conta níveis, folha tem altura 1, como nos heaps binários do curso). É a mesma árvore! Ao responder exercícios de “alturas” (lista, ex. 8 e 10), *declare a convenção*. Neste resumo: ordem = grau da raiz = k e altura = k arestas.

3.2 Heap binomial

Definição 3.3. Um **heap binomial** é uma floresta H de árvores binomiais tal que (1) cada árvore satisfaz a condição de heap (filho $\leq$ pai) e (2) as árvores têm **ordens distintas** (no máximo uma de cada ordem). As raízes ficam numa lista ordenada por ordem crescente.

Proposição 3.4. (a) Para cada n há um único formato de heap binomial com n nós: $B_k \in H$ sse o bit k de n é 1. (b) Logo H tem no máximo $\lfloor \log_2 n \rfloor + 1$ árvores, cada uma de altura $\leq \lfloor \log_2 n \rfloor$.

Demonstração. B_k contribui 2^k nós e as ordens são distintas, então $n = \sum_{B_k \in H} 2^k$ — é a representação binária de n , que é única. Um número n tem $\lfloor \log_2 n \rfloor + 1$ bits. $\square$

Ex.: $13 = 1101_2 \Rightarrow H = \{B_0, B_2, B_3\}$; $26 = 11010_2 \Rightarrow \{B_1, B_3, B_4\}$.

3.2.1 Operações

Binomial-Link e fusão (Szwarcfiter Alg. 9 / monitoria)

Procedimento BINOMIAL-LINK(y, z) % y e z raízes de B_{k-1} ; z fica por cima
 $p[y] \leftarrow z$; $sibling[y] \leftarrow child[z]$; $child[z] \leftarrow y$; $degree[z] \leftarrow degree[z] + 1$

Procedimento FUSÃO(H', H'')

$H := H' \cup H''$ % intercala as listas de raízes por ordem crescente: $O(\log n)$

enquanto H contém duas árvores B, B' de mesma ordem **faça** % a menor ordem repetida primeiro
se $B.raiz.chave \geq B'.raiz.chave$ **então**

subordinar B' a B
senão
subordinar B a B'

A fusão é exatamente **uma soma binária com “vai um”**: duas B_k viram uma B_{k+1} (o “vai um”). Como há $O(\log n)$ posições e no máximo 3 árvores de mesma ordem ao mesmo tempo (duas originais + um carry), a fusão custa $O(\log n)$ no pior caso. (Na implementação do Cormen, percorre-se a lista com ponteiros $prev-x$, x , $next-x$ e há 4 casos: ordens diferentes; três iguais seguidas; dois iguais com x vencendo; dois iguais com $next-x$ vencendo.)

Exemplo 3.5 (Fusão – Szwarcfiter Fig. 9.10 e exercício da monitoria). $H_1 = \{B_0, B_2, B_4\}$ ($21 = 10101_2$) e $H_2 = \{B_0, B_1\}$ ($3 = 11_2$). Links na ordem: $B_0 + B_0 \rightarrow B_1$; esse $B_1 + B_1$ (de H_2) $\rightarrow B_2$; esse $B_2 + B_2$ (de H_1) $\rightarrow B_3$. Resultado $\{B_3, B_4\}$, isto é, $24 = 11000_2 = 10101_2 + 11_2$.

- **Inserção:** fusão com um heap $\{B_0\}$ contendo só o novo nó. Pior caso $O(\log n)$ (quando $n = 2^k - 1$, todos os bits são 1 e há k links). *Tempo esperado* $O(1)$ (Lema 9.3 do Szwarcfiter: se j é a posição do primeiro 0 de n em binário, lida da direita para a esquerda, a inserção faz j “fusões” no sentido do livro, das quais $j - 1$ são links, um por 1 final de n ; com bits equiprováveis, $E[j] = \sum_{j \geq 1} j/2^j = 2$, isto é, 2 passos esperados e 1 link esperado) e *amortizado* $O(1)$ (dá-se 1 crédito a cada árvore; cada link é pago pelo crédito da árvore subordinada, que some — é o contador binário).
- **Seleção do máximo:** o máximo é uma das $\leq \lfloor \log n \rfloor + 1$ raízes: $O(\log n)$, ou $O(1)$ mantendo um ponteiro para a maior raiz (atualizado nas operações).
- **Remoção do máximo:** ache a raiz r máxima, numa árvore B_k ; retire B_k de H ($H' = H - B_k$); retirando r , seus filhos $B_{k-1}, \dots, B_0$ formam um heap binomial H'' ; faça a fusão $H' \cup H''$. Total $O(\log n)$.
- **Alteração de prioridade:** *aumentar* $\Rightarrow$ sobe dentro da própria árvore trocando com o pai, como no heap binário: $O(\log n)$ (altura $\leq \log n$). *Diminuir* $\Rightarrow$ remover o nó (ver abaixo) e reinseri-lo com a nova prioridade: $O(\log n)$. (Descer trocando com o maior filho também funciona, mas cada nível tem até $O(\log n)$ filhos para comparar: $O(\log^2 n)$.)
- **Remoção de um nó qualquer:** aumente sua prioridade para $+\infty$ (sobe até a raiz) e remova o máximo: $O(\log n)$.
- **Construção:** n inserções custam $O(n)$ no total (contador binário: $\sum_k n/2^k \leq 2n$). É a resposta do ex. 7 da lista.

3.3 Heap de Fibonacci

Ideia: preguiça. O heap binomial é “estrito”: toda inserção/fusão já resolve colisões de ordem. O de Fibonacci *adia* toda a arrumação para a remoção do máximo.

Estrutura. Floresta de árvores com condição de heap; as raízes numa **lista circular duplamente encadeada**, e os filhos de cada nó também. Ponteiro para a raiz máxima. Cada nó guarda: chave, pai, um filho, irmãos esquerdo/direito, grau, e um bit **marca**. Listas circulares duplas permitem inserir/remover um nó ou concatenar duas listas em $O(1)$.

Operações (heap de máximo, como no curso; “aumentar prioridade” = *decrease-key* do Cormen, que usa heap de mínimo).

- **Inserção:** cria uma árvore de um nó e a coloca na lista de raízes; atualiza o ponteiro do máximo. $O(1)$ real.
- **Fusão:** concatena as duas listas de raízes; máximo = maior dos dois. $O(1)$ real.
- **Seleção:** ponteiro. $O(1)$.

- **Remoção do máximo:** (1) remove a raiz máxima e promove todos os seus filhos a raízes (desmarcados); (2) **consolida:** percorre as raízes com um vetor $A[0..D]$ indexado pelo grau ($D = O(\log n)$); sempre que duas raízes têm o mesmo grau, subordina a de menor prioridade à de maior (o grau sobe de 1) e repete; (3) reconstrói a lista de raízes a partir de A e acha o novo máximo. Real: $O(D + t)$ com $t =$ número de raízes; **amortizado** $O(\log n)$.
- **Aumento de prioridade de x :** aumenta a chave; se não ficou maior que o pai, fim. Senão: **corta** x (com sua subárvore) e o põe na lista de raízes, desmarcado; então, subindo: se o pai y está *desmarcado*, marque-o (se não for raiz) e pare; se y já estava *marcado* (perdeu um filho antes), corte y também, desmarque-o, leve-o à lista de raízes e continue com o pai de y — **cortes em cascata**. **Amortizado** $O(1)$.
- **Remoção de nó qualquer:** aumenta para $+\infty$ e remove o máximo: $O(\log n)^*$.
- **Diminuição de prioridade:** não é a operação “barata”; faz-se removendo e reinserindo (ou cortando os filhos que passarem a violar): $O(\log n)^*$. Na tabela da aula (convenção de mínimo), isso aparece como “incrementar $O(\log n)^*$, decrementar $O(1)^*$ ”.

Regras das marcas. Todos começam desmarcados; um nó é marcado quando *perde o primeiro filho* por corte; ao perder o segundo, ele próprio é cortado. **Raízes nunca estão marcadas** (e podem perder vários filhos); ao ser subordinado a outro nó, o nó fica desmarcado.

Exemplo 3.6 (Cortes em cascata – exercício da monitoria de 26/08). Nó interno X com pai Y (marcado) e avô Z (marcado); a prioridade de X aumenta violando a ordem. (1) X é cortado e vai para a lista de raízes (desmarcado). (2) Y perdeu seu segundo filho: é cortado, desmarcado, vai para a lista de raízes. (3) Idem para Z . (4) Seja W o pai de Z : se W é raiz, nada muda; se não é raiz e está desmarcado, passa a **marcado** e a cascata para; se estava marcado, a cascata continua. Nós movidos para as raízes: X, Y, Z (e possivelmente mais); flags: X, Y, Z desmarcados, W marcado (se não raiz).

3.3.1 Por que funciona: grau máximo $O(\log n)$

Lema 3.7. *Seja x um nó e $y_1, \dots, y_k$ seus filhos na ordem em que foram ligados a x . Então $\text{grau}(y_i) \geq i - 2$.*

Demonstração. Quando y_i foi ligado, x já tinha $y_1, \dots, y_{i-1}$, logo $\text{grau} \geq i - 1$; só se ligam nós de mesmo grau, então y_i tinha $\text{grau} \geq i - 1$. Desde então y_i perdeu no máximo um filho (senão teria sido cortado). Logo $\text{grau}(y_i) \geq i - 2$. $\square$

Teorema 3.8. *Um nó de grau k tem subárvore com pelo menos $F_{k+2} \geq \varphi^k$ nós (F = Fibonacci, $\varphi = \frac{1+\sqrt{5}}{2}$). Logo o grau máximo é $D(n) \leq \log_\varphi n = O(\log n)$, e um heap de Fibonacci tem $O(\log n)$ árvores após a consolidação.*

Esboço. Seja s_k o tamanho mínimo de uma subárvore de raiz com grau k . Pelo lema, $s_k \geq 2 + \sum_{i=2}^k s_{i-2}$, e por indução $s_k \geq F_{k+2}$ (pois $F_{k+2} = 1 + \sum_{i=0}^k F_i$). É daí que vem o nome: as menores árvores de posto k ($F_0, \dots, F_5$ desenhadas em aula) têm 1, 2, 3, 5, 8, 13 nós. $\square$

Sem marcas/cascata, poderíamos cortar todos os netos de uma raiz: uma árvore de posto k com só $k + 1$ nós; o posto deixaria de ser $O(\log n)$ e a consolidação ficaria cara (resposta ao “Exercício 3” da monitoria).

3.3.2 Análise amortizada

Potencial (Cormen, cap. 19): $\Phi(H) = t(H) + 2m(H)$, com $t =$ nº de raízes e $m =$ nº de nós marcados.

- Inserção: real $O(1)$, $\Delta\Phi = +1 \Rightarrow O(1)$. Fusão: $\Delta\Phi = 0 \Rightarrow O(1)$.

- Remoção do máximo: real $O(D + t)$; depois há $\leq D + 1$ raízes, então $\Delta\Phi \leq (D + 1) - t \Rightarrow$ amortizado $O(D) = O(\log n)$. O custo de consolidar as t raízes acumuladas foi “pago” pelas inserções/fusões preguiçosas.
- Aumento de prioridade com c cortes em cascata: real $O(c)$; t cresce c , m cai $\geq c - 2 \Rightarrow \Delta\Phi \leq c + 2(2 - c) = 4 - c \Rightarrow$ amortizado $O(1)$. (Visão do banqueiro do Szwarcfiter: o corte de um nó marcado é debitado à operação que o marcou.)

Teorema 3.9 (Aula de 21/08). A partir de um heap de Fibonacci vazio, qualquer sequência de a_1 inserções, a_2 remoções, a_3 aumentos de prioridade e a_4 uniões custa $O(a_1 + a_3 + a_4 + a_2 \log n)$.

Armadilhas comuns

- Os $O(1)$ do Fibonacci são **amortizados**: uma remoção do máximo após muitas inserções/fusões pode custar $\Theta(n)$ (consolida tudo de uma vez). Se o requisito é previsibilidade de cada operação, Fibonacci é ruim.
- Constantes grandes e muita memória por nó (4 ponteiros + grau + marca): na prática só compensa com MUITAS alterações de prioridade (Dijkstra/Prim em grafos densos).
- Uma árvore de um heap de Fibonacci *pode* ter altura grande (até $\Theta(n)$ em sequências adversárias; Cormen, ex. 19.4-1): o que é logarítmico é o **grau**, não a altura.

3.4 Comparação e escolha da estrutura (Q4 e Teste 1)

	Heap binário	Heap binomial	Heap de Fibonacci
Seleção	$O(1)$	$O(\log n)$ / $O(1)$ c/ ponteiro	$O(1)$
Inserção	$O(\log n)$	$O(\log n)$ pior; $O(1)$ esp./amort.	$O(1)^*$
Remoção do máx.	$O(\log n)$	$O(\log n)$	$O(\log n)^*$
Aumentar prioridade	$O(\log n)$	$O(\log n)$	$O(1)^*$
Fusão	$O(n)$	$O(\log n)$ pior caso	$O(1)^*$
Construção	$O(n)$	$O(n)$	$O(n)$
Memória	vetor, sem ponteiros	3 ponteiros + grau/nó	4 ponteiros + grau + marca/nó
Garantia	pior caso	pior caso	amortizada
Simplicidade	muito simples	média	complexa

Roteiro de escolha

1. Precisa de **fusão**? Não $\Rightarrow$ **heap binário** (mais simples, menos memória, ótimo em cache).
2. Fusão frequente e **cada operação precisa ser previsível** (limite de tempo por operação, sistema de tempo real) $\Rightarrow$ **heap binomial** (fusão $O(\log n)$ *pior caso*).
3. Muitíssimos **aumentos de prioridade** (ou fusões) e o que importa é o **custo total** da execução $\Rightarrow$ **heap de Fibonacci**.
4. Restrição de **memória** forte $\Rightarrow$ binário (as outras têm overhead de ponteiros por nó).

Teste 1 – respostas esperadas.

- **Q1 (aeroporto)**: fusões muito frequentes com filas de centenas de milhares de aeronaves; alterações raras; exigência de tempo *previsível por operação individual*. $\Rightarrow$ **Heap binomial**: fusão em $O(\log n)$ no pior caso de *cada* execução, crescendo só logaritmicamente. Binário: fusão $O(n)$ (reconstrução) — milhares de fusões com 10^5 elementos estouram a janela. Fibonacci: fusão $O(1)$ só *amortizada*; o custo acumulado aparece de uma vez numa remoção (consolidação

$\Theta(n)$ possível) — imprevisível; e sua vantagem (aumento de prioridade $O(1)$) não é usada, pois alterações são raras.

- **Q2 (central médica, 40 MB):** $2 \cdot 10^6$ chamadas $\times$ 16 bytes = 32 000 000 bytes $\approx$ 30,5 MB $<$ 41 943 040. Heap binomial precisaria de ≥ 3 ponteiros (pai, filho, irmão = 24 bytes) + grau: ≥ 40 bytes/nó $\Rightarrow \geq 80 \cdot 10^6$ bytes; Fibonacci (≥ 4 ponteiros + grau + marca) ainda mais. Não há fusão, alterações são poucas, inserção/remoção $O(\log n)$ bastam. $\Rightarrow$ **Heap binário em vetor** (único que cabe, e o mais simples).
- **Q3 (supercomputador):** $5 \cdot 10^6$ inserções, $5 \cdot 10^6$ remoções, $80 \cdot 10^6$ aumentos de urgência, poucas uniões; importa o custo total. Fibonacci: $\approx 5 \cdot 10^6 + 80 \cdot 10^6 + 5 \cdot 10^6 \log_2(5 \cdot 10^6) \approx 2 \cdot 10^8$ passos; binário/binomial: $\approx 90 \cdot 10^6 \cdot 22 \approx 2 \cdot 10^9$. $\Rightarrow$ **Heap de Fibonacci** (aumento $O(1)^*$ sem remover/reinserir). Binário/binomial pagam $O(\log n)$ em cada um dos 80 milhões de aumentos.

4 Conjuntos disjuntos: Union-Find

4.1 O problema

Manter uma partição de um universo de n elementos em conjuntos disjuntos, cada um identificado por um **representante**, com:

- **MAKE-SET(x):** cria o conjunto $\{x\}$ (Szwarcfiter: *gerar*);
- **FIND(x):** devolve o representante do conjunto de x (*buscar*); “ x e y estão juntos?” $\iff$ $\text{FIND}(x) = \text{FIND}(y)$;
- **UNION(x, y):** une os conjuntos de x e y (*fundir*).

Conjuntos **nunca se separam** (não há “split”). Uma sequência típica: n MAKE-SET e m FIND/UNION intercalados; no máximo $n - 1$ uniões efetivas. O número de conjuntos = número de raízes, que nunca aumenta. **Aplicações:** componentes conexas dinâmicas, algoritmo de Kruskal (AGM), equivalência de símbolos em compiladores, grupos em redes sociais, percolação.

4.2 Implementações

(a) Vetor de representantes (“quick-find”)

$\text{rep}[x]$ guarda o representante. $\text{FIND} = O(1)$; UNION precisa trocar rep de todos os elementos de um conjunto: $O(n)$ por união.

(b) Listas encadeadas com ponteiro direto ao representante

Cada conjunto é uma lista; um objeto-conjunto guarda *head*, *tail* e *size*; todo nó tem ponteiro para o objeto-conjunto (o representante).

- **MAKE-SET:** $O(1)$. **FIND:** $O(1)$ (segue um ponteiro).
- **UNION:** concatena as listas em $O(1)$ (tail de uma $\rightarrow$ head da outra) *mas* precisa atualizar o ponteiro ao representante de todos os nós da lista anexada.
- Sem cuidado: $\Theta(n)$ por união, $\Theta(n^2)$ numa sequência ruim (sempre anexar a lista grande à unitária).
- **União ponderada** (anexa a *menor* à maior): cada vez que o ponteiro de um elemento x é atualizado, o conjunto de x pelo menos **dobra**; como tamanhos $\leq n$, x é atualizado $\leq \log_2 n$ vezes. Total de n MAKE-SET e m operações: $O(m + n \log n)$, isto é, UNION em $O(\log n)$ **amortizado**.
- Vantagem exclusiva: **listar os membros** de um conjunto em $O(k)$ (k = tamanho do conjunto), percorrendo a lista.

(c) Floresta (“quick-union”)

Cada conjunto é uma árvore; cada nó aponta para o pai ($\text{pai}[x]$); a raiz é o representante ($\text{pai}[r] = r$ ou 0). MAKE-SET: $\text{pai}[x] := x$. FIND: sobe até a raiz, $O(h)$. UNION: acha as duas raízes e faz uma apontar para a outra, $O(1)$ *dadas as raízes*. Problema: sem critério, uma sequência de uniões gera uma **cadeia de altura** $n - 1$ (Find $\Theta(n)$).

(d) União por tamanho ou por rank

Liga-se a raiz da árvore *menor* (em tamanho, ou em *rank*) à raiz da maior. **Rank** (aula de 25/08):

- todo nó começa com rank 0;
- só o rank de raízes é atualizado: na união de raízes de ranks *iguais* r , a nova raiz passa a ter rank $r + 1$; com ranks diferentes, nada muda;
- o rank nunca decresce; quando um nó deixa de ser raiz, seu rank nunca mais muda;
- sem compressão de caminhos, rank = altura; com compressão, rank é só uma cota superior da altura.

Teorema 4.1 (Altura logarítmica). *Com união por rank (ou por tamanho), toda árvore de rank/altura h tem pelo menos 2^h nós. Logo $h \leq \lfloor \log_2 n \rfloor$ e FIND custa $O(\log n)$.*

Demonstração. Indução no número de operações. Uma árvore de um nó tem $h = 0$ e $2^0 = 1$ nó. A altura (rank) de uma árvore só aumenta quando se unem duas de mesmo rank h : pela hipótese cada uma tem $\geq 2^h$ nós, e a nova tem $\geq 2 \cdot 2^h = 2^{h+1}$ nós e rank $h + 1$. Ligando a menor sob a maior, a altura não aumenta. Como $2^h \leq n$, $h \leq \log_2 n$. (Szwarcfiter, Teo. 8.2, com união por tamanho e altura contada em níveis: $2^{h(T)-1} \leq |T|$, usando $h(T) = \max\{1 + h(T_v), h(T_w)\}$ com $|T_v| \leq |T_w|$.) $\square$

Corolário 4.2. (i) *O número de nós com rank exatamente k é $\leq n/2^k$ (cada um é raiz, em algum momento, de uma subárvore com $\geq 2^k$ nós, e essas subárvores são disjuntas).* (ii) *Todo rank é $\leq \lfloor \log_2 n \rfloor$.* (iii) *$\text{rank}(x) < \text{rank}(\text{pai}[x])$ para todo x não raiz.*

(e) Compressão de caminhos

Durante FIND(x), depois de achar a raiz, faz-se *todo nó do caminho* apontar diretamente para a raiz. As próximas buscas desses nós custam $O(1)$. Variantes (Szwarcfiter): *separação* de caminhos (cada nó passa a apontar para o avô) e *divisão* (nós alternados apontam para o avô) — mesma complexidade assintótica, uma só passada.

Floresta com união por rank e compressão de caminhos

Procedimento MAKE-SET(x)

$\text{pai}[x] := x; \text{rank}[x] := 0$

Função FIND(x)

se $\text{pai}[x] \neq x$ **então**

$\text{pai}[x] := \text{FIND}(\text{pai}[x])$

% compressão

retorna $\text{pai}[x]$

Procedimento UNION(x, y)

$r := \text{FIND}(x); s := \text{FIND}(y)$

se $r = s$ **então retorna**

se $\text{rank}[r] < \text{rank}[s]$ **então**

$\text{pai}[r] := s$

senão se $\text{rank}[r] > \text{rank}[s]$ **então**

$\text{pai}[s] := r$

senão

$\text{pai}[s] := r; \text{rank}[r] := \text{rank}[r] + 1$

Versão iterativa da compressão (Szwarcfiter, *comp-buscar*): uma passada acha a raiz y ; uma segunda passada percorre de novo o caminho fazendo $\text{pai}[\cdot] := y$.

4.3 Rank + compressão: $O(m \log^* n)$ (demonstração da aula de 25/08)

$\log^* n$ = número de vezes que se aplica $\log_2$ até chegar a ≤ 1 . Defina $F(0) = 1$, $F(k) = 2^{F(k-1)}$: $F = 1, 2, 4, 16, 65536, 2^{65536}$. Então $\log^* n$ é o menor k com $F(k) \geq n$ ($\log^* n \leq 5$ para qualquer n prático).

Grupos de ranks: o grupo $B \geq 1$ contém os ranks em $(F(B-1), F(B)]$, e o grupo 0 os ranks $\leq F(0) = 1$: grupo 0 = $\{0, 1\}$, 1 = $\{2\}$, 2 = $\{3, 4\}$, 3 = $\{5, \dots, 16\}$, 4 = $\{17, \dots, 65536\}$.

- **Propriedade A:** como ranks $\leq \log n$, há no máximo $\log^* n$ grupos (ranks até $\log n$ caem no grupo $\leq \log^* n - 1$).
- **Propriedade B:** o número de nós no grupo B é $\leq \sum_{r=F(B-1)+1}^{F(B)} \frac{n}{2^r} \leq \frac{n}{2^{F(B-1)}} = \frac{n}{F(B)}$ (para $B \geq 1$; o grupo 0 tem trivialmente $\leq n = n/F(0)$ nós).

Num FIND, cobre-se cada nó x do caminho: se x é raiz, filho da raiz, ou se $\text{pai}[x]$ está num grupo diferente do de x , cobra-se do FIND — como os ranks crescem ao longo do caminho, há $O(\log^* n)$ mudanças de grupo por FIND: total $O(m \log^* n)$. Caso contrário ($\text{pai}[x]$ no mesmo grupo), cobra-se de x : a compressão dá a x um novo pai de rank *estritamente maior*; um nó do grupo B só pode receber isso $\leq F(B)$ vezes antes que seu pai vá para outro grupo (e aí nunca mais é cobrado). Total por grupo: $\leq \frac{n}{F(B)} \cdot F(B) = n$; somando os $\log^* n$ grupos, $O(n \log^* n)$. **Total:** $O(m \log^* n)$ (com $m \geq n$).

Deslocamento na lista de grupos das notas

Nas notas de 25/08 a lista explícita “grupo 0: $\{0\}$, grupo 1: $\{1\}$, grupo 2: $\{2\}$, grupo 3: $\{3, 4\}$, grupo 4: $\{5, \dots, 16\}$ ” está deslocada de uma posição em relação à fórmula $(F(B-1), F(B)]$ escrita logo acima, que dá grupo 1 = $(1, 2] = \{2\}$, grupo 2 = $(2, 4] = \{3, 4\}$ etc. Nada muda na análise: continua havendo $O(\log^* n)$ grupos e a Propriedade B continua valendo. Na prova, use a fórmula e seja consistente. As notas escrevem m tanto para o número de elementos quanto para o de operações; aqui usamos n para elementos e m para operações.

Observação 4.3. O limite justo (Tarjan) é $O(m \alpha(m, n))$, com α a inversa da função de Ackermann ($\alpha \leq 4$ na prática; Szwarcfiter §8.4: $O(n + m \alpha(m + n, n))$). Para a prova, o resultado demonstrado em aula é o $O(m \log^* n)$; ambos são “quase lineares”.

4.4 Qual implementação usar? (Q2 da AV1)

Tabela 1: Implementações de Union-Find e quando usar cada uma.

Implementação	MAKE-SET	FIND	UNION	Quando é a melhor escolha
Vetor de representantes	$O(1)$	$O(1)$	$O(n)$	Quase só consultas e pouquíssimas uniões, n pequeno.
Listas encadeadas + ponteiro ao representante + união ponderada	$O(1)$	$O(1)$ pior caso	$O(\log n)^*$	Consultas dominam e cada uma precisa ser $O(1)$ <i>garantido</i> ; é preciso listar os membros de um grupo em $O(k)$; memória extra aceitável (head/tail/size + ponteiros).
Floresta ingênua	$O(1)$	$O(n)$	$O(1) + \text{Finds}$	Nunca (só didática).
Floresta + união por rank/tamanho (sem compressão)	$O(1)$	$O(\log n)$	$O(\log n)$	Quando é preciso desfazer uniões (rollback/“split” offline): sem compressão, cada união muda só um ponteiro (e talvez um rank), que se empilha e se desfaz em $O(1)$.
Floresta + rank + compressão de caminhos	$O(1)$	$O(\log^* n)^* (\alpha)$		Caso geral, muitas operações misturadas, só <i>Find</i> e <i>Union</i> (Kruskal, conectividade, redes sociais): praticamente $O(1)$ por operação, memória mínima (um vetor <i>pai</i> + <i>rank</i>).

Cenários típicos (documento “Conjuntos Disjuntos” do Classroom)

- **“Listar os membros do grupo de U ”:** a floresta só tem ponteiros filho→pai; listar exigiria testar $\text{FIND}(i) = \text{FIND}(U)$ para todos os N elementos ($O(N)$). Com **listas encadeadas**: percorre-se a lista, $O(k)$; *MesmoGrupo* em $O(1)$; *Juntar* em $O(\log n)^*$ (menor na maior).
- **Remoção de arestas / conectividade após destruições** (“A queda do império”): Union-Find não desfaz uniões (e a compressão destrói a topologia). Solução *offline*: leia todas as consultas, construa o estado final (sem as arestas que serão destruídas) e processe **de trás para frente**, transformando cada destruição numa UNION; guarde as respostas e imprima invertido.
- **Operações online com desfazer (snapshot/rollback): sem compressão**, só união por rank/tamanho (altura $O(\log n)$, FIND $O(\log n)$); cada UNION empilha o ponteiro/rank alterado; desfazer = desempilhar e restaurar em $O(1)$.
- **Kruskal**: ordena arestas; para cada aresta (u, v) , se $\text{FIND}(u) \neq \text{FIND}(v)$ inclui e faz UNION. Floresta com rank + compressão.

Teste 1, Q4 – plataforma com 10^7 usuários

10^5 uniões e 10^8 consultas “mesmo grupo?”; grupos nunca se separam; não importa o histórico. (a) Estrutura: **Union-Find**. Entre “árvore apontando até o representante” e “lista com ponteiro direto ao representante”: as consultas dominam ($10^8 \gg 10^5$) e precisam ser rápidas; na **lista encadeada** cada FIND é $O(1)$ *no pior caso* (um acesso), e o custo fica nas raras uniões ($O(\log n)^*$ com união ponderada: no total $\leq n \log n \approx 10^7 \cdot 23$ atualizações, diluídas). **Resposta recomendada: a lista encadeada**, porque com razão consultas:uniões de $10^8 : 10^5$ o que importa é o custo de cada FIND, e a lista o garante em $O(1)$ *no pior caso*. A floresta com rank + compressão também é defensável ($O(\alpha)^*$ por operação e menos memória), desde que se explique que a garantia é amortizada; o que *não* se deve propor é floresta sem balanceamento. (b) Estratégia de união: **anexar o menor ao maior** (união por tamanho na lista; por rank/tamanho na árvore). Na lista, garante que cada elemento muda de representante $\leq \log_2 n$ vezes (o tamanho do seu grupo dobra a cada mudança). Na árvore, garante altura $\leq \log_2 n$ (Teo. 4.1), evitando

cadeias de altura $\Theta(n)$ que tornariam cada consulta $\Theta(n)$.

4.5 Exercícios resolvidos (monitoria de revisão e lista)

Exemplo 4.4 (Revisão, Q2). Partindo de $\{A\}, \dots, \{F\}$ e as uniões $\text{UNION}(A, B)$, $\text{UNION}(C, D)$, $\text{UNION}(A, C)$, $\text{UNION}(E, F)$, com união por rank e, em empate, a primeira raiz vence. $\text{UNION}(A, B)$: ranks $0 = 0 \Rightarrow$ raiz A , $\text{rank}(A) = 1$. $\text{UNION}(C, D)$: raiz C , $\text{rank}(C) = 1$. $\text{UNION}(A, C)$: $1 = 1 \Rightarrow$ raiz A , $\text{rank}(A) = 2$ (árvore $A \rightarrow \{B, C\}$, $C \rightarrow D$). $\text{UNION}(E, F)$: raiz E , $\text{rank}(E) = 1$. Representantes finais: A (de $\{A, B, C, D\}$) e E (de $\{E, F\}$): **2 conjuntos**.

Exemplo 4.5 (Revisão, Q3 e Q4). Cadeia $\text{pai}[B] = A$, $\text{pai}[C] = B$, $\text{pai}[D] = C$, $\text{pai}[E] = D$. $\text{FIND}(E)$ com compressão devolve A e deixa B, C, D, E todos apontando para A (árvore de altura 1). Vantagem: consultas futuras a esses nós custam $O(1)$. Se $\text{rank}(A) = \text{rank}(D) = 1$ e $\text{UNION}(A, D)$: empate, a nova raiz é A (pela convenção) e $\text{rank}(A) = 2$. O rank só sobe no empate porque, ligando a menor sob a maior, a altura da maior não muda; só com alturas iguais a nova árvore fica 1 mais alta.

Exemplo 4.6 (Revisão, Q5 – por que usar as duas técnicas juntas?). Só rank, sem compressão: FIND fica $\Theta(\log n)$ (há sequências que atingem essa altura). Só compressão, sem rank: uma única árvore pode ficar alta (cadeia) antes de ser comprimida; custo total $O(m \log n)$ (Tarjan–van Leeuwen), e FIND individual pode custar $\Theta(n)$. Juntas: $O(m \log^* n)$ (ou $O(m \alpha)$) no total — praticamente constante por operação.

Exemplo 4.7 (Lista UF/AVL, ex. 1 – a cota $\lfloor \log n \rfloor$ é justa). A prova é o Teo. 4.1. Sequência que atinge altura $\log_2 n$ ($n = 2^k$): una os elementos aos pares (altura 1), depois as árvores aos pares (altura 2), etc. Após k rodadas há uma árvore de rank/altura $k = \log_2 n$ (é uma árvore binomial $B_k!$).

Exemplo 4.8 (Lista UF/AVL, ex. 2 – só compressão). Árvore alta antes da compressão: $\text{UNION}(1, 2)$, $\text{UNION}(1, 3)$, ..., sempre pendurando a raiz da árvore grande sob o novo elemento unitário ($\text{pai}[\text{raiz}] := i$): forma uma cadeia $1 \rightarrow 2 \rightarrow \dots \rightarrow n$ de altura $n - 1$, sem nenhum FIND caro no caminho. Um $\text{FIND}(1)$ custa $\Theta(n)$ e achata tudo. Resultado conhecido: com compressão e união arbitrária, m operações custam $O(m \log n)$ (na verdade $\Theta(m \log_{1+m/n} n)$); a ideia é que cada compressão que passa por um nó “leve” (cuja subárvore é menos da metade da do pai) só pode acontecer $O(\log n)$ vezes por nó.

Pontos-chave

Q2 em uma frase: **listas encadeadas** dão $\text{FIND } O(1)$ pior caso, $\text{UNION } O(\log n)^*$ com “menor na maior” e listagem de membros em $O(k)$; **florestas** com **rank + compressão** dão $O(\log^* n)$ (ou α) amortizado para tudo com memória mínima; **sem compressão** se for preciso desfazer uniões. Sempre **menor no maior**, senão árvores/listas degeneram.

5 Árvores binárias de busca e árvores AVL

5.1 Árvore binária de busca (ABB)

Motivação: lista sequencial ordenada dá busca binária $\Theta(\log n)$, mas inserção/remoção $\Omega(n)$; lista encadeada insere rápido mas busca em $\Omega(n)$. A ABB combina busca eficiente com estrutura dinâmica.

Definição 5.1. Árvore binária em que, para todo nó p : a chave de p é $\geq$ às chaves da subárvore esquerda e $\leq$ às da subárvore direita. (O percurso *em ordem* lista as chaves ordenadas.)

- **Busca:** desce comparando; custo $\Theta(h)$, h = altura.

- **Inserção:** busca a posição e insere como **folha**: $\Theta(h)$.
- **Remoção** de p : (i) folha: remove; (ii) um filho: o filho ocupa o lugar de p ; (iii) dois filhos: o **maior da subárvore esquerda** (predecessor: desce à esquerda uma vez e depois sempre à direita) assume o lugar de p (alternativa simétrica: menor da direita). $\Theta(h)$.
- $\lfloor \log_2 n \rfloor + 1 \leq h \leq n$ (em níveis). Inserções em ordem crescente/decrescente geram uma “lista” de altura $n \Rightarrow$ tudo vira $\Theta(n)$.

Árvores completas minimizam a altura, mas mantê-las completas após inserções custa $\Omega(n)$ (reorganização global). Solução: exigir só **balanceamento**: altura $O(\log n)$, e o mesmo para toda subárvore (m nós $\Rightarrow$ altura $O(\log m)$), com rebalanceamento em $O(\log n)$.

5.2 Árvores AVL (Adel’son-Vel’skii e Landis, 1962)

Definição 5.2. Uma árvore binária é **AVL** se, para todo nó v , as alturas das subárvores esquerda e direita diferem de no máximo 1 (v é *regulado*; senão, *desregulado*). Define-se

$$\text{bal}(v) = h_D(v) - h_E(v) \quad (\text{direita menos esquerda; bal} = -1 \text{ “pesa à esquerda”}),$$

e v é regulado sse $\text{bal}(v) \in \{-1, 0, 1\}$. Folhas têm $\text{bal} = 0$. Toda árvore completa é AVL (a recíproca é falsa).

Teorema 5.3 (AVL é balanceada). *Toda árvore AVL com n nós tem altura $O(\log n)$ (precisamente $h < 1,44 \log_2(n + 2)$).*

Demonstração. Seja $N(h)$ o número mínimo de nós de uma AVL de altura h (em níveis). $N(1) = 1$, $N(2) = 2$ e, para $h \geq 3$, uma AVL mínima de altura h tem raiz, uma subárvore de altura $h - 1$ e a outra de altura $h - 2$ (a menor possível), ambas mínimas:

$$N(h) = 1 + N(h - 1) + N(h - 2).$$

(Essas árvores mínimas são as **árvores de Fibonacci**; $N(h) = F_{h+2} - 1$.) Como N é crescente, $N(h) > 2N(h - 2)$, donde $N(h) \geq 2^{\lfloor h/2 \rfloor}$, isto é, $h \leq 2 \log_2 N(h) + 1$. Como qualquer AVL de altura h com n nós tem $n \geq N(h)$, $h = O(\log n)$. $\square$

Consequência: busca em $O(\log n)$. Falta mostrar que inserção e remoção, com o rebalanceamento, também são $O(\log n)$.

5.3 Inserção: atualização dos balanços

Insere-se como numa ABB (a nova chave vira folha; se já existe, para). Depois percorre-se o caminho de volta, do pai w do novo nó até a raiz, atualizando bal . Suponha que a inserção foi na subárvore **esquerda** de p (o caso direito é simétrico, trocando os sinais):

$\text{bal}(p)$ antes	depois	altura de $T(p)$	ação
+1	0	não mudou	para (ancestrais não mudam)
0	-1	aumentou 1	continua com o pai de p (se p é a raiz, para)
-1	-2	—	p desregulado: rotação ; depois, para

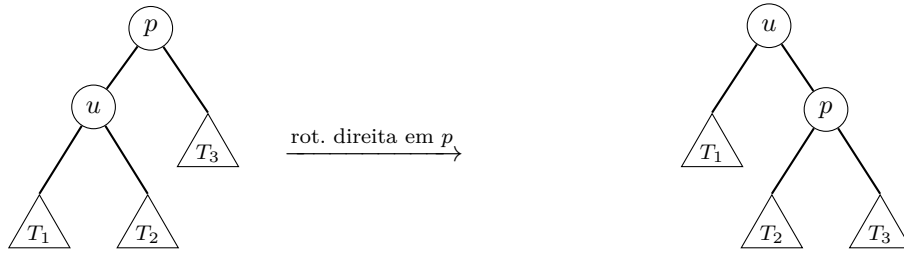
Uma inserção aumenta alturas em no máximo 1, logo o desregulado tem $|\text{bal}| = 2$. O p a rotacionar é o **primeiro nó desregulado encontrado ao subir da folha para a raiz** (o mais profundo). Como a rotação devolve à subárvore a altura que ela tinha *antes* da inserção, **uma inserção faz no máximo uma rotação** (simples ou dupla).

5.4 As rotações

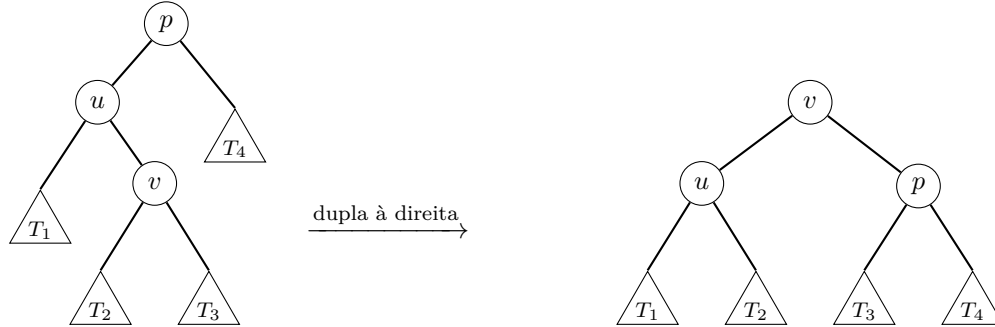
Seja p o nó desregulado mais profundo.

Caso	$\text{bal}(p)$	filho do lado pesado	sinais	rotação
1.1 (“LL”)	-2	$u = \text{esq.}, \text{bal}(u) = -1$	iguais	simples à direita em p
1.2 (“LR”)	-2	$u = \text{esq.}, \text{bal}(u) = +1$	opostos	dupla à direita : esq. em u , dir. em p
2.1 (“RR”)	+2	$z = \text{dir.}, \text{bal}(z) = +1$	iguais	simples à esquerda em p
2.2 (“RL”)	+2	$z = \text{dir.}, \text{bal}(z) = -1$	opostos	dupla à esquerda : dir. em z , esq. em p

Regra mnemônica (monitoria): **FB positivo** $\Rightarrow$ **rotações à esquerda**; **negativo** $\Rightarrow$ **à direita**.
Mesmo sinal: simples; **sinais opostos: dupla**. (Filho com $\text{bal} = 0$ só ocorre na *remoção*, e aí usa-se rotação simples.)



Caso 1.1: $h(T_1) = h + 1$, $h(T_2) = h(T_3) = h$. Depois: $\text{bal}(u) = \text{bal}(p) = 0$; altura volta a $h + 2$.



Caso 1.2: v sobe para a raiz. Depois $\text{bal}(v) = 0$; se $\text{bal}(v)$ era -1 : $\text{bal}(u) = 0$, $\text{bal}(p) = +1$; se era $+1$: $\text{bal}(u) = -1$, $\text{bal}(p) = 0$; se era 0 (v é o novo nó): ambos 0 .

Os casos 2.1 e 2.2 são espelhados (rotação simples à esquerda: z sobe, p vira filho esquerdo de z e o antigo filho esquerdo de z vira filho direito de p). **Rotações preservam a ordem da ABB** (o percurso em ordem T_1, u, T_2, p, T_3 é o mesmo) e custam $O(1)$ (só ponteiros).

5.5 Pseudocódigo (slides do professor)

Cada nó tem campos **esq**, **dir**, **chave**, **bal**; λ é o ponteiro nulo; $pt \uparrow .\text{campo}$ acessa o campo do nó apontado. Chamada externa: $\text{INS-AVL}(x, p_{\text{raiz}})$. O booleano h devolvido indica se **a altura da subárvore aumentou** (só então o pai precisa atualizar seu balanço).

Inserção

Procedimento INICIAR-NO(pt, x)

alocar(pt); $pt \uparrow .\text{esq} := \lambda$; $pt \uparrow .\text{dir} := \lambda$; $pt \uparrow .\text{chave} := x$; $pt \uparrow .\text{bal} := 0$

Função INS-AVL(x , ref pt)

% retorna true se a altura de $T(pt)$ aumentou

se $pt = \lambda$ então

INICIAR-NO(pt, x); **retorna** true

se $x = pt \uparrow .\text{chave}$ então

retorna false

% chave repetida

se $x < pt \uparrow .\text{chave}$ então

```

     $h := \text{INS-AVL}(x, pt \uparrow .esq)$ 
    se  $h$  então
         $pt, h := \text{ATUALIZABALANÇO}(pt, \text{"esquerda"})$ 
    senão
         $h := \text{INS-AVL}(x, pt \uparrow .dir)$ 
        se  $h$  então
             $pt, h := \text{ATUALIZABALANÇO}(pt, \text{"direita"})$ 
    retorna  $h$ 
Função  $\text{ATUALIZABALANÇO}(pt, a)$ 
     $h := \text{false}$ 
    se  $a = \text{"esquerda"}$  então
        se  $pt \uparrow .bal = 1$  então
             $pt \uparrow .bal := 0$ 
        senão se  $pt \uparrow .bal = 0$  então
             $pt \uparrow .bal := -1; h := \text{true}$  % altura aumentou: o pai verifica
        senão
             $pt := \text{ROTAÇÕESDIREITA}(pt)$  % era -1: ficaria -2
    senão
        se  $pt \uparrow .bal = -1$  então
             $pt \uparrow .bal := 0$ 
        senão se  $pt \uparrow .bal = 0$  então
             $pt \uparrow .bal := 1; h := \text{true}$ 
        senão
             $pt := \text{ROTAÇÕESESQUERDA}(pt)$ 
    retorna  $(pt, h)$  % se houve rotação,  $pt$  aponta para a nova raiz da subárvore

```

Rotações (slides)

```

Função  $\text{ROTAÇÕESDIREITA}(pt)$ 
     $ptu := pt \uparrow .esq$ 
    se  $ptu \uparrow .bal = -1$  então % simples à direita
         $pt \uparrow .esq := ptu \uparrow .dir; ptu \uparrow .dir := pt; pt \uparrow .bal := 0; ptu \uparrow .bal := 0; pt := ptu$ 
    senão % dupla à direita
         $ptv := ptu \uparrow .dir$ 
         $ptu \uparrow .dir := ptv \uparrow .esq; ptv \uparrow .esq := ptu; pt \uparrow .esq := ptv \uparrow .dir; ptv \uparrow .dir := pt$ 
        se  $ptv \uparrow .bal = -1$  então  $pt \uparrow .bal := 1$ 
        senão  $pt \uparrow .bal := 0$ 
        se  $ptv \uparrow .bal = 1$  então  $ptu \uparrow .bal := -1$ 
        senão  $ptu \uparrow .bal := 0$ 
         $ptv \uparrow .bal := 0; pt := ptv$ 
    retorna  $pt$ 
Função  $\text{ROTAÇÕESESQUERDA}(pt)$ 
     $ptz := pt \uparrow .dir$ 
    se  $ptz \uparrow .bal = 1$  então % simples à esquerda
         $pt \uparrow .dir := ptz \uparrow .esq; ptz \uparrow .esq := pt; pt \uparrow .bal := 0; ptz \uparrow .bal := 0; pt := ptz$ 
    senão % dupla à esquerda
         $pty := ptz \uparrow .esq$ 
         $ptz \uparrow .esq := pty \uparrow .dir; pty \uparrow .dir := ptz; pt \uparrow .dir := pty \uparrow .esq; pty \uparrow .esq := pt$ 
        se  $pty \uparrow .bal = 1$  então  $pt \uparrow .bal := -1$ 
        senão  $pt \uparrow .bal := 0$ 
        se  $pty \uparrow .bal = -1$  então  $ptz \uparrow .bal := 1$ 
        senão  $ptz \uparrow .bal := 0$ 
         $pty \uparrow .bal := 0; pt := pty$ 
    retorna  $pt$ 

```

Detalhes do pseudocódigo que costumam ser perguntados

- **Para que serve h ?** Indica se a altura da subárvore cresceu; só nesse caso o pai precisa atualizar o balanço. Por isso ATUALIZABALANÇO só é chamada se $h = \text{true}$: se a altura não mudou, nenhum ancestral muda de balanço.
- **Por que atualizar pt após a rotação?** A raiz da subárvore mudou (u, v, z ou y sobe); como pt é passado *por referência*, a atribuição religa a nova raiz ao pai (ou atualiza $ptraz$).
- Nos slides, os três testes de ATUALIZABALANÇO aparecem como “se” consecutivos; eles precisam ser **mutuamente exclusivos** (“senão se”), como acima. Com “se” em sequência, um nó com $\text{bal} = 1$ viraria 0 e, logo em seguida, o teste “ $\text{bal} = 0$ ” dispararia de novo (bug do “aluno” da monitoria).
- Após a rotação na inserção, $h := \text{false}$ (a altura voltou ao valor antigo).

5.6 Remoção em AVL

Remove-se como numa ABB (se p tem dois filhos, o maior da subárvore esquerda o substitui) e sobe-se a partir do nó *fisicamente removido* atualizando balanços. Agora a pergunta é se a altura **diminuiu**. Remoção na subárvore **direita** de p (esquerda é simétrico):

$\text{bal}(p)$ antes	depois	altura de $T(p)$	ação
+1	0	diminuiu	continua com o pai
0	-1	não mudou	para
-1	-2	—	rotação à direita (ver abaixo)

Na rotação, olha-se o filho esquerdo u :

- $\text{bal}(u) = -1$: simples à direita; ambos ficam 0; a altura **diminuiu** $\Rightarrow$ continua subindo.
- $\text{bal}(u) = 0$ (*caso exclusivo da remoção*): simples à direita; fica $\text{bal}(p) = -1$, $\text{bal}(u) = +1$; a altura **não muda** $\Rightarrow$ para.
- $\text{bal}(u) = +1$: dupla à direita; altura **diminuiu** $\Rightarrow$ continua subindo.

Por isso a remoção pode precisar de uma rotação em **cada** ancestral: $O(\log n)$ rotações, cada uma $O(1) \Rightarrow O(\log n)$ no total.

Armadilhas comuns

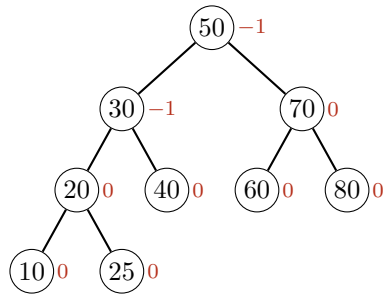
Os slides implementam a remoção em dois passos (REMOVER-AVL-PASSO1: remoção de ABB que devolve o nó mais profundo afetado e o ajuste $+1/0/-1$; REMOVER-AVL-PASSO2: sobe atualizando com ATUALIZABALANÇOAPÓSREMOÇÃO). Ao reutilizar ROTAÇÕESDIREITA na remoção, o caso $ptu \uparrow .\text{bal} = 0$ cairia na rotação *dupla* (o teste é só “ $= -1$ ”), o que está errado: com $\text{bal}(u) = 0$ deve-se fazer a **simples** (teste “ ≤ 0 ”) e ajustar $\text{bal}(p) = -1$, $\text{bal}(u) = +1$, devolvendo “altura não diminuiu”.

Exemplo 5.4 (Remoções pequenas (conferidas)).

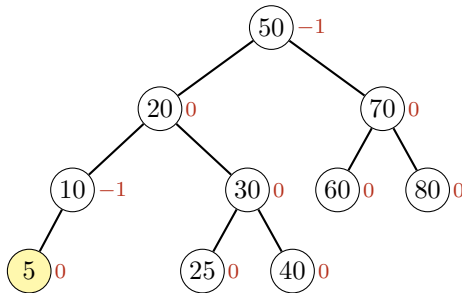
- $20^{+1}(10, 40^0(30, 50))$, remover 10: $\text{bal}(20) = +2$, filho 40 com $\text{bal} = 0 \Rightarrow$ simples à esquerda em 20: $40^{-1}(20^{+1}(\cdot, 30), 50)$; altura inalterada.
- $20^{-1}(10^{+1}(\cdot, 15), 30)$, remover 30: $\text{bal}(20) = -2$, filho 10 com $\text{bal} = +1 \Rightarrow$ dupla à direita: $15(10, 20)$.

5.7 Exemplos resolvidos (conferidos por simulação)

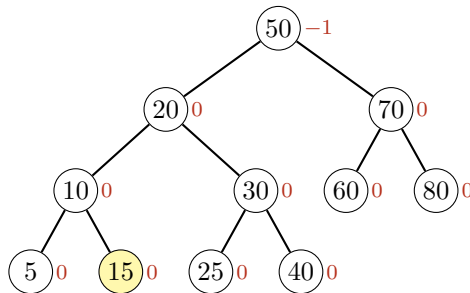
Exemplo 5.5 (Teste 1, Q5). Árvore inicial e inserções 5, 15, 35, 45 (balanços em vermelho, $\text{bal} = h_D - h_E$):



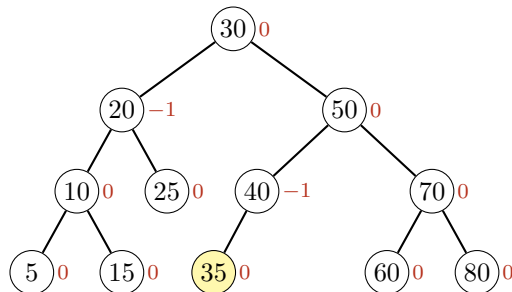
Inserir 5 (filho esq. de 10). Subindo: $\text{bal}(10) = -1$, $\text{bal}(20) = -1$, $\text{bal}(30) = -2$. **Primeiro desregulado: 30**; filho esq. 20 com $\text{bal} = -1$ (mesmo sinal) $\Rightarrow$ caso 1.1, **rotação simples à direita em 30**. A altura da subárvore volta a 3, então 50 fica com -1 .



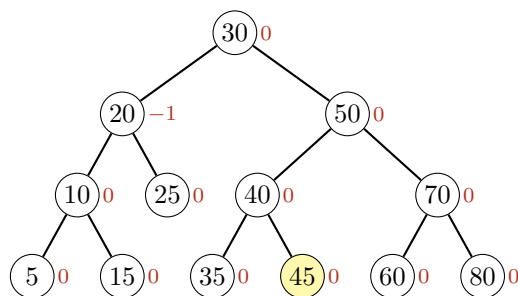
Inserir 15 (filho dir. de 10): $\text{bal}(10)$ vai de -1 a 0 e a altura de $T(10)$ não muda: **para**. Sem rotação.



Inserir 35 (filho esq. de 40). Subindo: $\text{bal}(40) = -1$, $\text{bal}(30) = +1$, $\text{bal}(20) = +1$, $\text{bal}(50) = -2$. **Primeiro desregulado: 50**; filho esq. 20 com $\text{bal} = +1$ (sinais opostos) $\Rightarrow$ caso 1.2, **rotação dupla à direita**: esquerda em 20, depois direita em 50. O nó $v = 30$ sobe para a raiz; como $\text{bal}(30)$ era $+1$: $\text{bal}(20) = -1$, $\text{bal}(50) = 0$.



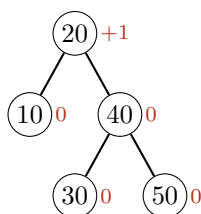
Inserir 45 (filho dir. de 40): $\text{bal}(40)$ vai de -1 a 0 , altura de $T(40)$ inalterada: **para**. Árvore final:



(c) Numa ABB comum, inserir códigos em ordem desfavorável (crescente, decrescente, “zigue-zague”) gera altura $\Theta(n)$ e busca/inserção/remoção em $\Theta(n)$; a AVL garante altura $\leq 1,44 \log_2 n$ (Teo. 5.3), logo $O(\log n)$ independentemente da ordem de chegada, pagando só $O(1)$ rotações por inserção.

Exemplo 5.6 (Monitoria de revisão, Q5: 40, 20, 10, 30, 35). 40; $40^{-1}(20)$; inserir 10: $\text{bal}(40) = -2$, filho 20 com $-1 \Rightarrow$ simples à direita em 40: $20^0(10, 40)$. Inserir 30: $20^{+1}(10, 40^{-1}(30, \cdot))$. Inserir 35 (dir. de 30): $\text{bal}(30) = +1$, $\text{bal}(40) = -2 \Rightarrow$ primeiro desregulado é **40**, filho esq. 30 com $+1 \Rightarrow$ **dupla à direita** (esq. em 30, dir. em 40). Final: $20^{+1}(10^0, 35^0(30^0, 40^0))$. Rotações: na 3ª inserção (simples, direita) e na 5ª (dupla, direita).

Exemplo 5.7 (Monitoria de revisão, Q7: 10, 20, 30, 40, 50 — e um erro no slide). Inserir 30: $\text{bal}(10) = +2$, filho 20 com $+1 \Rightarrow$ simples à esquerda em 10: $20(10, 30)$. Inserir 40: $20^{+1}(10, 30^{+1}(\cdot, 40))$. Inserir 50: subindo, $\text{bal}(40) = +1$, $\text{bal}(30) = +2 \Rightarrow$ o **primeiro desregulado é 30** (e não 20!): simples à esquerda em 30. **Final correto:** $20^{+1}(10^0, 40^0(30^0, 50^0))$.



Atenção: o slide da monitoria rotaciona em 20 e chega a uma árvore com raiz 30 — está errado: a rotação é feita no desregulado *mais profundo*; após rotacionar em 30, o nó 20 fica com $\text{bal} = +1$ e nem chega a desregular.

Inserir 49 na árvore correta: vai para a esquerda de 50; $\text{bal}(50) = -1$, $\text{bal}(40) = +1$, $\text{bal}(20) = +2 \Rightarrow$ desregulado 20, filho dir. 40 com $+1 \Rightarrow$ **simples à esquerda em 20**: $40^0(20^0(10, 30), 50^{-1}(49, \cdot))$. (Partindo da árvore errada do slide, $30(20(10), 40(\cdot, 50))$, o desregulado seria 40 com filho 50 de $\text{bal} = -1$, e aí sim seria a **dupla à esquerda** mostrada no slide, com resultado $30(20(10), 49(40, 50))$.)

5.8 Exercícios da lista (Union-Find e AVL)

Exemplo 5.8 (Ex. 3 – remoção com rotação em *todos* os ancestrais). Use uma **árvore de Fibonacci** (AVL mínima) em que todo nó interno tem $\text{bal} = -1$ (esquerda mais alta): T_h tem raiz, T_{h-1} à esquerda e T_{h-2} à direita. Removendo a **folha mais à direita** (caminho raiz $\rightarrow T_{h-2} \rightarrow T_{h-4} \rightarrow \dots$), cada ancestral perde altura do lado já mais baixo, fica com $\text{bal} = -2$ e tem filho esquerdo com $\text{bal} = -1 \Rightarrow$ rotação simples que *diminui* a altura, propagando o problema. Exemplo conferido: T_7 com 33 nós (chaves 1..33 em ordem), raiz 21; remover a folha 33 (ancestrais 32, 29, 21) causa rotações em **32, 29 e 21**. Em geral, h ímpar dá $(h-1)/2$ rotações $= \Theta(\log n)$.

Exemplo 5.9 (Ex. 4 – fusão de duas AVL (alto nível)). Caso “junção” (todas as chaves de $T_1 <$ todas as de T_2): retire o mínimo x de T_2 (ou o máximo de T_1) em $O(\log n)$. Suponha $h(T_1) \geq h(T_2)$: desça pela espinha direita de T_1 até um nó v com $h(v) \in \{h(T_2), h(T_2) + 1\}$; crie o nó x com filhos $T(v)$ e T_2 e coloque-o no lugar de v ; suba rebalanceando como numa inserção (no máximo uma rotação ajusta tudo). Custo $O(|h_1 - h_2| + 1) = O(\log n)$. Caso geral (chaves intercaladas): percursos

em ordem de ambas ($O(n + m)$), intercalação das duas listas ordenadas ($O(n + m)$) e construção de uma árvore completa a partir da lista ordenada (sempre a mediana como raiz, $O(n + m)$) — total linear. Alternativa: inserir cada nó da menor na maior, $O(m \log(n + m))$.

Exemplo 5.10 (Ex. 5 – AVL como estrutura de conjuntos disjuntos?). Possível: cada conjunto é uma AVL e cada elemento guarda ponteiro para o pai. $\text{FIND}(x)$ sobe até a raiz da sua AVL (que identifica o conjunto): $O(\log n)$. UNION : inserir os elementos da menor AVL na maior, $O(k \log n)$ (k = tamanho da menor); com “menor na maior”, cada elemento é reinserido $\leq \log n$ vezes, total $O(n \log^2 n)$. Ganha-se ordem (percurso ordenado, busca por chave), mas é pior que o Union-Find ($\approx O(1)$ amortizado) para FIND/UNION .

Heap $\times$ AVL $\times$ ABB (revisão, Q6 e Q14)

- **Fila de prioridade pura** (sempre atender o de maior prioridade): **heap** — máximo em $O(1)$, inserir/remover $O(\log n)$, vetor simples. A AVL também faria tudo em $O(\log n)$, mas não dá o máximo em $O(1)$ sem ponteiro extra e é mais pesada.
- **Busca por chave arbitrária, remoção de elemento específico, percurso em ordem, sucessor/predecessor, intervalos**: **AVL** (o heap não tem ordem entre irmãos: buscar uma chave é $O(n)$).
- **ABB simples $\times$ AVL**: com inserções/remoções frequentes e ordem de chegada imprevisível, **AVL** (ABB pode degenerar para $\Theta(n)$).

5.9 Árvores rubro-negras (pesquisa)

Complemento – pesquisa proposta em 02/09, entrega 19/09 (Cormen, cap. 13)

Definição: ABB em que cada nó é *vermelho* ou *preto* (1 bit extra) e: (1) a raiz é preta; (2) as folhas nulas (λ) são pretas; (3) um nó vermelho tem os dois filhos pretos (não há dois vermelhos seguidos); (4) todo caminho de um nó até as folhas nulas descendentes tem o mesmo número de nós pretos (*altura negra bh*).

Por que é balanceada: por (4), a subárvore de um nó x tem $\geq 2^{bh(x)} - 1$ nós internos (indução); por (3), pelo menos metade dos nós de qualquer caminho raiz-folha é preta, então $bh(\text{raiz}) \geq h/2$. Logo $n \geq 2^{h/2} - 1$, isto é, $h \leq 2 \log_2(n + 1)$.

Inserção: insere como ABB e pinta o nó z de *vermelho* (não altera alturas negras); se o pai é vermelho, viola (3). Correção olhando o *tio y*: (caso 1) tio vermelho $\Rightarrow$ recolore pai e tio de preto e avô de vermelho, e continua do avô; (caso 2) tio preto e z em “zigue-zague” $\Rightarrow$ rotação no pai para virar caso 3; (caso 3) tio preto e z “em linha” $\Rightarrow$ rotação no avô e troca de cores pai/avô. No fim, raiz preta. No máximo **2 rotações** por inserção; recolorações $O(\log n)$.

Remoção: remoção de ABB; se o nó retirado era preto, surge um “preto duplo” que é resolvido olhando o irmão (4 casos), com no máximo **3 rotações**.

AVL $\times$ rubro-negra: AVL é mais rigidamente balanceada ($h \leq 1,44 \log n$ vs. $2 \log n$): buscas um pouco mais rápidas. Rubro-negra faz menos rotações em atualizações (remoção $O(1)$ rotações vs. $O(\log n)$ na AVL): preferida em bibliotecas (`std::map`, `TreeMap`, kernel Linux).

Pontos-chave

AVL: $bal = h_D - h_E \in \{-1, 0, 1\}$; altura $\leq 1,44 \log n$ (árvores de Fibonacci). Inserção: sobe atualizando; $+1 \rightarrow 0$ para; $0 \rightarrow \pm 1$ continua; $\pm 1 \rightarrow \pm 2$ rotaciona no **mais profundo** e para (1 rotação). Sinais iguais: simples; opostos: dupla. Remoção: pode rotacionar em todos os ancestrais; filho com $bal = 0 \Rightarrow$ simples e para.

6 Ordenação

Problema: dado S , obter uma permutação $s_1, \dots, s_n$ com $s_i \leq s_{i+1}$.

- **Estável:** elementos de chaves iguais aparecem na saída na mesma ordem relativa da entrada (essencial para o radix sort e para ordenar por várias chaves).
- **In-place:** memória adicional independente de n (ou $O(\log n)$ de pilha). Quicksort e heapsort são in-place; mergesort e counting sort não.

6.1 Ordenação por comparação

Mergesort (intercalação)

MergeSort e Intercalação

Procedimento MERGESORT(V, ini, fim)

se $ini < fim$ então

$meio := \lfloor (ini + fim)/2 \rfloor$

 MERGESORT($V, ini, meio$); MERGESORT($V, meio + 1, fim$)

 INTERCALA($V, ini, meio, fim$)

Procedimento INTERCALA($V, ini, meio, fim$)

$i := ini; j := meio + 1; k := 1$

% W = vetor auxiliar

 enquanto $i \leq meio$ e $j \leq fim$ faça

 se $V[i] \leq V[j]$ então

$W[k] := V[i]; i := i + 1$

% “ $\leq$ ” mantém a estabilidade

 senão

$W[k] := V[j]; j := j + 1$

$k := k + 1$

 copie o que sobrou de $V[i..meio]$ ou de $V[j..fim]$ para o fim de W

 copie $W[1..fim - ini + 1]$ de volta para $V[ini..fim]$

$T(n) = 2T(n/2) + \Theta(n) \Rightarrow \Theta(n \log n)$ em todos os casos. **Estável** (com “ $\leq$ ”), **não in-place** ($\Theta(n)$ extra). Versão iterativa (*bottom-up*): intercale blocos de tamanho 1, 2, 4, ... ao longo do vetor.

Quicksort

Se $n \leq 1$, pronto; escolha um **pivô** x ; particione em $S_1 = \{w < x\}$ e $S_2 = \{w \geq x\}$; ordene recursivamente e concatene.

Quicksort com partição de Lomuto

Procedimento QUICKSORT(V, ini, fim)

se $ini < fim$ então

$q := \text{PARTICIONA}(V, ini, fim)$

 QUICKSORT($V, ini, q - 1$); QUICKSORT($V, q + 1, fim$)

Função PARTICIONA(V, ini, fim)

$x := V[fim]; i := ini - 1$

% pivô no fim (troque antes se escolher outro)

 para $j := ini$ até $fim - 1$ faça

 se $V[j] < x$ então

$i := i + 1; V[i] \Leftrightarrow V[j]$

$V[i + 1] \Leftrightarrow V[fim]$; retorna $i + 1$

- **Pior caso** $\Theta(n^2)$: partições sempre 0 e $n - 1$ (ex.: pivô = primeiro elemento e vetor já ordenado ou invertido): $T(n) = T(n - 1) + \Theta(n)$.
- **Melhor/médio** $\Theta(n \log n)$. Argumento do slide: com pivô “aleatório”, em metade dos casos a parte maior tem tamanho $\leq 3n/4$; se isso sempre ocorresse, no nível t os subproblemas teriam tamanho $\leq (3/4)^t n$, $O(\log n)$ níveis de custo $O(n)$; como ocorre com probabilidade 1/2, em média a cada 2 níveis o tamanho cai por 3/4: ainda $O(n \log n)$ esperado.

- **Escolha do pivô:** primeiro elemento (ruim se já ordenado), aleatório, **mediana de três** (primeiro, meio, último). Usando a **mediana exata via BFPRT** (§7) obtém-se $O(n \log n)$ no *pior* caso, mas a constante é alta — na prática não compensa (resposta a um exercício dos slides).
- **In-place** (partição por trocas; recursão $O(\log n)$ de pilha se recursar primeiro na parte menor). **Não estável** (a troca de longa distância embaralha iguais); dá para torná-lo estável com partição estável usando $O(n)$ de memória extra.

Heapsort

Ver §2: $\Theta(n \log n)$ sempre, in-place, não estável.

Algoritmos quadráticos (para referência)

Inserção: insere $V[i]$ na parte já ordenada $V[1..i-1]$ deslocando os maiores; $\Theta(n^2)$ pior, $\Theta(n)$ se já ordenado, estável, in-place, ótimo para n pequeno ou quase ordenado (usado dentro do bucket sort).
Bolha: $\Theta(n^2)$, estável.

6.2 Limite inferior $\Omega(n \log n)$ para ordenação por comparação

Teorema 6.1. *Todo algoritmo de ordenação baseado em comparações faz $\Omega(n \log n)$ comparações no pior caso.*

Demonstração. Modela-se o algoritmo (para cada n) por uma **árvore de decisão**: cada nó interno é uma comparação $a_i \leq a_j?$, com dois filhos (os dois resultados); cada folha é um estado final (uma permutação que ordena a entrada). A altura h é o número de comparações no pior caso. Como qualquer uma das $n!$ permutações pode ser a resposta, há pelo menos $n!$ folhas; uma árvore binária de altura h tem no máximo 2^h folhas. Logo

$$2^h \geq n! \geq n(n-1) \cdots \left(\frac{n}{2}\right) \geq \left(\frac{n}{2}\right)^{n/2} \Rightarrow h \geq \frac{n}{2} \log_2 \frac{n}{2} = \frac{n}{2} \log n - \frac{n}{2} = \Omega(n \log n). \quad \square$$

Corolário 6.2. *Heapsort e mergesort são **ótimos** entre os algoritmos por comparação (supondo comparação $O(1)$). Para ordenar em $o(n \log n)$ é preciso não se basear em comparações: usar a estrutura das chaves (inteiros pequenos, dígitos, distribuição).*

6.3 Ordenação em tempo linear

Resposta ao desafio “dá para ordenar em tempo linear?”: depende da natureza dos elementos.

Counting sort (inteiros em $[0, k]$)

COUNTING-SORT(A, n, k) (slides)

```

para  $i := 0$  até  $k$  faça
     $C[i] := 0$ 
para  $j := 1$  até  $n$  faça
     $C[A[j]] := C[A[j]] + 1$ 
    %  $C[i] = \#\{j : A[j] = i\}$ 
para  $i := 1$  até  $k$  faça
     $C[i] := C[i] + C[i-1]$ 
    %  $C[i] = \#\{j : A[j] \leq i\}$ 
para  $j := n$  decrescendo até 1 faça
     $B[C[A[j]]] := A[j]; C[A[j]] := C[A[j]] - 1$ 
    % de trás para frente  $\Rightarrow$  estável
retorna  $B$ 

```

- Tempo e espaço $\Theta(n + k)$; **linear** se $k = O(n)$. Se $k = \Theta(2^n)$ fica exponencial: é **pseudopolinomial**.

- **Estável:** a última ocorrência de cada valor vai para a última posição do seu bloco. **Não in-place** (vetores B e C). Não faz nenhuma comparação entre elementos.

Radix sort (inteiros com d dígitos na base b)

Ordena **dígito a dígito, do menos significativo (LSD) para o mais significativo**, usando em cada passada um método **estável**.

Radix sort com filas (slides)

```

para  $i := 1$  até  $d$  faça
  para  $j := 1$  até  $n$  faça
     $k := i$ -ésimo dígito menos significativo de  $L[j]$  na base  $b$ ; insere  $L[j]$  no fim da fila  $F[k]$ 
   $j := 1$ 
  para  $k := 0$  até  $b - 1$  faça
    enquanto  $F[k]$  não vazia faça
      remove o início de  $F[k]$  e põe em  $L[j]$ ;  $j := j + 1$ 

```

- **Por que precisa ser estável?** Depois de ordenar pelo dígito i , elementos com o mesmo dígito i devem manter a ordem obtida pelos dígitos $1..i - 1$; é o que garante (por indução) que após a passada i a lista está ordenada pelos i últimos dígitos. As filas preservam a ordem naturalmente (alternativa: counting sort por dígito).
- Tempo $\Theta(d(n + b))$; com b e d constantes, $\Theta(n)$. Truque: inteiros até n^c na base $b = n$ têm $d = c$ dígitos $\Rightarrow \Theta(cn)$.
- **Exemplo da aula (conferido):** 102, 313, 690, 131, 222, 670, 690, 130, 079, 102.
 Unidades: 690 670 690 130 131 102 222 102 313 079;
 dezenas: 102 102 313 222 130 131 670 079 690 690;
 centenas: 079 102 102 130 131 222 313 670 690 690.
- Chaves de tamanhos variáveis d_i (exercício): agrupe por número de dígitos (counting por tamanho) e aplique o radix de modo que cada número só participe das passadas dos seus próprios dígitos, ou use o MSD/American Flag recursivo: $O(\sum d_i)$. Para **strings**: ordenar da direita para a esquerda completando as menores com um símbolo “menor que todos”, ou MSD (primeiro caractere) recursivo.

Bucket sort (reais uniformes em $[0, 1)$)

Divide $[0, 1)$ em r baldes iguais, distribui ($A[i]$ vai para o balde $\lfloor r A[i] \rfloor$), ordena cada balde (ex.: inserção) e concatena. Com entrada uniforme, cada balde tem em média n/r elementos; com $r = \Theta(n)$ o tempo **esperado** é $\Theta(n)$ (Cormen: $E[n_i^2] = 2 - 1/n$, então $\sum E[O(n_i^2)] = O(n)$). Pior caso: o do algoritmo usado nos baldes (tudo num balde só).

American Flag sort

Radix **MSD in-place**: conta quantos elementos há de cada dígito (vetor C de tamanho b), calcula onde começa cada bloco e *permuta no próprio vetor* cada elemento para o seu bloco (trocas); depois recursão em cada bloco no dígito seguinte. **Memória auxiliar**: só os contadores, $O(b)$ por nível de recursão, $O(b \cdot d)$ no total (independe de n) $\Rightarrow$ **in-place**. **Não é estável** (as trocas mudam a ordem de iguais).

6.4 Qual algoritmo de ordenação usar?

Situação	Escolha
Inteiros num intervalo pequeno ($k = O(n)$)	Counting sort $\Theta(n + k)$
Inteiros/IDs com poucos dígitos (d fixo), k enorme	Radix sort $\Theta(d(n + b))$
Reais uniformemente distribuídos em um intervalo	Bucket sort $\Theta(n)$ esperado
Chaves arbitrárias, precisa de garantia e/ou estabilidade	Mergesort $\Theta(n \log n)$ (memória $\Theta(n)$)
Chaves arbitrárias, memória escassa, garantia	Heapsort $\Theta(n \log n)$ in-place
Chaves arbitrárias, desempenho médio na prática	Quicksort (pivô aleatório/mediana de 3)
Quase ordenado ou n pequeno	Inserção
Dados em lista encadeada / arquivos externos	Mergesort

Teste 2, Exercício 3 – respostas esperadas

- **A** (10^7 notas entre 0 e 100): **Counting sort**. $k = 101 \ll n$: $\Theta(n + k) = \Theta(n)$, vetor de contagem de 101 posições; comparação daria $\Omega(n \log n) \approx 2,3 \cdot 10^8$ comparações.
- **B** (10^6 identificadores de 12 dígitos): **Radix sort**. Counting é inviável ($k = 10^{12}$); radix com $d = 12$ e $b = 10$ faz 12 passadas lineares: $\Theta(d(n + b)) \approx 1,2 \cdot 10^7$ (ou base $10^3/10^4$: 4 ou 3 passadas).
- **C** (10^5 reais uniformes em $[0, 1)$): **Bucket sort**, n baldes, tempo esperado $\Theta(n)$ (radix/-counting não se aplicam a reais).
- **D** (10^6 inteiros em $[-10^9, 10^9]$): **Radix sort** após somar 10^9 (chaves em $[0, 2 \cdot 10^9]$, ≈ 31 bits: 2 passadas na base 2^{16} , ou 10 na base 10). Counting inviável ($k = 2 \cdot 10^9 \gg n$ de memória). Mergesort/quicksort $\Theta(n \log n)$ também seriam aceitáveis, mas o radix é linear.
- **E** (10^5 registros, chaves arbitrárias, sem restrição de domínio): só **comparação**: **Mergesort** (garantia $\Theta(n \log n)$ e estabilidade, ideal para registros) ou Quicksort (mais rápido na média, in-place, mas $\Theta(n^2)$ no pior caso e não estável).

6.5 Q4: dado um código, qual estrutura e qual ordenação?

A Q4 dará um código genérico; o roteiro é: **(1)** liste as operações que o código faz repetidamente; **(2)** estime quantas vezes cada uma ocorre; **(3)** escolha a estrutura que torna barata a operação dominante; **(4)** olhe o tipo das chaves para escolher a ordenação; **(5)** justifique com complexidades (e memória/estabilidade quando relevante).

O código faz repetidamente...	Estrutura
pega o maior/menor, insere, (às vezes) altera prioridade	heap binário
... e une filas com frequência (previsível)	heap binomial
... com muitíssimos aumentos de prioridade (custo total)	heap de Fibonacci
“ x e y estão no mesmo grupo?” / une grupos	Union-Find (rank + compressão)
busca por chave, inserção, remoção, em ordem, sucessor, mínimo/máximo, intervalos	AVL (ou rubro-negra)
só busca/inserção/remoção por chave exata, sem ordem	tabela hash ($O(1)$ esperado)
muitas buscas, quase nenhuma atualização	vetor ordenado + busca binária
k -ésimo menor / mediana uma vez	seleção linear (BFPRT/quickselect), sem ordenar
os k maiores de um fluxo	heap de mínimo de tamanho k : $O(n \log k)$

Padrões a reconhecer em código: laço que procura o máximo de uma lista e o remove ($\Theta(n)$ por iteração $\Rightarrow$ heap); laço que ordena a lista inteira a cada inserção ($\Rightarrow$ heap ou AVL); laço que

percorre todos os elementos para saber se dois estão no mesmo grupo ($\Rightarrow$ Union-Find); ordenar para depois pegar só o k -ésimo ($\Rightarrow$ seleção $O(n)$); ordenar inteiros pequenos com **sort** genérico ($\Rightarrow$ counting/radix).

Pontos-chave

Comparação $\Rightarrow \Omega(n \log n)$ (árvore de decisão, $2^h \geq n!$). Merge: estável, $\Theta(n)$ extra. Quick: in-place, médio $n \log n$, pior n^2 . Heap: in-place, sempre $n \log n$, não estável. Counting $\Theta(n + k)$ estável; Radix $\Theta(d(n + b))$ precisa de estável por dígito (LSD); Bucket $\Theta(n)$ esperado com uniforme.

7 Seleção: o algoritmo BFPRT (mediana das medianas)

Problema da seleção: dado um conjunto A de n números e k , achar o k -ésimo menor. Casos: mínimo ($k = 1$), máximo ($k = n$), mediana inferior $\lfloor (n + 1)/2 \rfloor$, superior $\lceil (n + 1)/2 \rceil$.

Possibilidades. (1) Ordenar: $O(n \log n)$. (2) Retirar o mínimo k vezes: $O(kn)$ ($O(n^2)$ para a mediana). (3) **Quickselect** (pivoteamento como no quicksort, mas recursão em *um* lado só): $O(n)$ esperado, $O(n^2)$ no pior caso. (4) **BFPRT** (Blum, Floyd, Pratt, Rivest, Tarjan): $O(n)$ no **pior caso**.

Quickselect (exercício do slide: tempo esperado $O(n)$, mais simples)

```

Função QUICKSELECT( $S, k$ )
    escolha um pivô  $m$  aleatório em  $S$ 
     $S_1 := \{s < m\}$ ;  $S_2 := \{s = m\}$ ;  $S_3 := \{s > m\}$ 
    se  $|S_1| \geq k$  então
        retorna QUICKSELECT( $S_1, k$ )
    senão se  $|S_1| + |S_2| \geq k$  então
        retorna  $m$ 
    senão
        retorna QUICKSELECT( $S_3, k - |S_1| - |S_2|$ )

```

Esperado: com probabilidade $\geq 1/2$ o pivô cai no “meio” (entre os quartis) e o lado mantido tem $\leq 3n/4$. Agrupe as chamadas em *fases*: a fase termina quando o tamanho cai para $\leq 3n/4$. O número de chamadas numa fase é dominado por uma geométrica de parâmetro $1/2$, logo tem esperança ≤ 2 , e cada chamada custa $\leq cn$. Assim $E[T(n)] \leq E[T(3n/4)] + 2cn$, e somando a série geométrica, $E[T(n)] \leq 2cn \sum_{t \geq 0} (3/4)^t = 8cn = O(n)$.

7.1 O algoritmo

A ideia é garantir um pivô *sempre* bom: a **mediana das medianas** de grupos de 5.

Seleção(S, k) – BFPRT (slides)

```

Função SELEÇÃO( $S, k$ )
    se  $|S| \leq 49$  então
        ordene  $S$  (mergesort) e retorna o  $k$ -ésimo menor
    divida  $S$  em  $\lceil |S|/5 \rceil$  grupos de (até) 5 elementos
    determine a mediana de cada grupo (ordenar 5 elementos:  $O(1)$  cada)
     $M :=$  conjunto das medianas;  $m :=$  SELEÇÃO( $M, \lceil |M|/2 \rceil$ )    % recursão 1: tamanho  $n/5$ 
     $S_1 := \{s \in S : s < m\}$ ;  $S_2 := \{s \in S : s = m\}$ ;  $S_3 := \{s \in S : s > m\}$ 
    se  $|S_1| \geq k$  então
        retorna SELEÇÃO( $S_1, k$ )    % recursão 2: tamanho  $\leq 3n/4$ 
    senão se  $|S_1| + |S_2| \geq k$  então
        retorna  $m$ 
    senão
        retorna SELEÇÃO( $S_3, k - |S_1| - |S_2|$ )

```

retorna SELEÇÃO(S_3 , $k - |S_1| - |S_2|$)

Armadilhas comuns

Num dos slides o teste aparece como “se $|S_1| = k$ então o k -ésimo está em S_1 ”; o correto é $|S_1| \geq k$ (como no pseudocódigo da página seguinte). E, ao ir para S_3 , o índice muda para $k - |S_1| - |S_2|$.

7.2 Análise

Lema 7.1. $|S_1| \leq n - 3\lfloor n/10 \rfloor$ e $|S_3| \leq n - 3\lfloor n/10 \rfloor$; para $n \geq 50$, isso é $\leq 3n/4$.

Demonstração. Das $\lfloor n/5 \rfloor$ medianas, pelo menos metade, $\lfloor \lfloor n/5 \rfloor / 2 \rfloor = \lfloor n/10 \rfloor$, são $\geq m$. Cada uma dessas vem de um grupo que tem mais 2 elementos $\geq$ ela (logo $\geq m$). Assim, pelo menos $3\lfloor n/10 \rfloor$ elementos são $\geq m$, ou seja, $|S_2| + |S_3| \geq 3\lfloor n/10 \rfloor$ e $|S_1| \leq n - 3\lfloor n/10 \rfloor$. Simetricamente para S_3 . Para $n - 3\lfloor n/10 \rfloor \leq 3n/4$ basta $12\lfloor n/10 \rfloor \geq n$, isto é, $2\lfloor n/10 \rfloor \geq n - 10\lfloor n/10 \rfloor$; como o lado direito é ≤ 9 , basta $\lfloor n/10 \rfloor \geq 5$, ou seja, $n \geq 50$ (por isso o caso base é $n \leq 49$). $\square$

Teorema 7.2. $T(n) \leq 20cn$, logo a BFPRT é $O(n)$ no pior caso.

Demonstração. $T(n) = O(1)$ para $n \leq 49$ e $T(n) \leq T(n/5) + T(3n/4) + cn$ para $n > 49$ (a partição e as medianas dos grupos são lineares). Por indução forte (com a base ajustada, p.ex. $T(n) \leq 6n \leq 20cn$ pelo mergesort para $n < 50$):

$$T(n) \leq 20c \frac{n}{5} + 20c \frac{3n}{4} + cn = 4cn + 15cn + cn = 20cn. \quad \square$$

Por que grupos de 5? Para linearidade, a soma dos tamanhos dos subproblemas precisa ser uma fração < 1 de n : $\frac{n}{5} + \frac{3n}{4} = \frac{19n}{20} < n$. Com grupos de 3: pivô garante só $\approx n/3$ de cada lado, $T(n) = T(n/3) + T(2n/3) + cn$ e $\frac{1}{3} + \frac{2}{3} = 1 \Rightarrow \Theta(n \log n)$. Grupos de 7, 9, ... também funcionam (constantes diferentes).

Aplicação. Quicksort com a mediana (BFPRT) como pivô: $T(n) = 2T(n/2) + O(n) = O(n \log n)$ no **pior** caso; na prática é mais lento que o quicksort aleatório por causa das constantes.

Pontos-chave

Seleção em $O(n)$: grupos de 5, mediana das medianas como pivô, $\geq 3\lfloor n/10 \rfloor$ de cada lado, $T(n) \leq T(n/5) + T(3n/4) + cn \leq 20cn$. Não é preciso ordenar para achar o k -ésimo!

8 Divisão e conquista

Paradigma: (1) *Divisão*: quebrar o problema Π em subproblemas menores **do mesmo tipo** $\Pi_1, \dots, \Pi_k$; (2) *Conquista*: resolvê-los recursivamente (casos pequenos diretamente); (3) *Combinação*: juntar as soluções parciais com um procedimento “simples”. Busca-se **bom balanceamento** (partes de tamanhos parecidos). Funciona bem quando os subproblemas são **disjuntos**; quando eles se repetem muito, a técnica certa é programação dinâmica (§9.2). Exemplos do curso: mergesort, quicksort, BFPRT, contagem de inversões, Strassen, subvetor máximo.

Q5: modelo de resposta para “escreva o pseudocódigo por divisão e conquista”

1. **Assinatura:** RESOLVE(V, ini, fim) trabalhando num intervalo (evita copiar vetores). Diga o que a função **retorna** (às vezes mais de um valor: soma e índices; contagem e vetor ordenado).
2. **Caso base:** $ini = fim$ (ou $n \leq$ constante): resposta direta.

3. **Divisão:** $meio := \lfloor (ini + fim)/2 \rfloor$.
4. **Conquista:** duas chamadas recursivas, $[ini, meio]$ e $[meio + 1, fim]$.
5. **Combinação:** o que “atravessa” o meio (a parte criativa); geralmente $O(n)$.
6. **Complexidade:** escreva a recorrência e resolva (árvore de recursão ou Teorema Mestre, §1.4): $2T(n/2) + \Theta(n) = \Theta(n \log n)$.
7. **Correção** (uma frase): toda solução está inteira à esquerda, inteira à direita ou cruza o meio; cada caso é tratado.

8.1 Contagem de inversões (aula de 11/09)

Dada $a_1, \dots, a_n$, um par $i < j$ é uma **inversão** se $a_i > a_j$ (métrica de similaridade entre rankings: minha ordem $1, \dots, n$ vs. a sua). Força bruta: $\Theta(n^2)$. **D&C:** divide em metades A e B ; conta recursivamente as inversões dentro de cada uma; **combina** contando os pares (a, b) com $a \in A$, $b \in B$, $a > b$. Se A e B estão *ordenadas*, isso é fácil durante a intercalação: ao comparar a_i e b_j , se $a_i \leq b_j$, a_i não é invertido com nenhum elemento restante de B ; se $a_i > b_j$, então b_j é invertido com **todos os elementos restantes de A** . Por isso a função também devolve o vetor ordenado (é um mergesort “instrumentado”).

Contagem de inversões (aula + completado)

```

Função CALCULAINVERSÕES( $V, ini, fim$ )                                % conta e deixa  $V[ini..fim]$  ordenado
    se  $ini = fim$  então
        retorna 0
     $meio := \lfloor (ini + fim)/2 \rfloor$ 
     $a := \text{CALCULAINVERSÕES}(V, ini, meio)$ 
     $b := \text{CALCULAINVERSÕES}(V, meio + 1, fim)$ 
     $c := \text{MERGECALCULANDOINVERSÕES}(V, ini, meio, fim)$ 
    retorna  $a + b + c$ 

Função MERGECALCULANDOINVERSÕES( $V, ini, meio, fim$ )
     $i := ini; j := meio + 1; k := 1; cont := 0$ 
    enquanto  $i \leq meio$  e  $j \leq fim$  faça
        se  $V[i] \leq V[j]$  então
             $W[k] := V[i]; i := i + 1$ 
        senão
             $W[k] := V[j]; j := j + 1$ 
             $cont := cont + (meio - i + 1)$                                 %  $V[j]$  é menor que todos os restantes da esquerda
         $k := k + 1$ 
    copie os restantes de  $V[i..meio]$  e de  $V[j..fim]$  para  $W$ ; copie  $W$  para  $V[ini..fim]$ 
    retorna  $cont$ 

```

$T(n) = 2T(n/2) + \Theta(n) \Rightarrow \Theta(n \log n)$. *Cuidado:* usar “ $\leq$ ” (iguais não são inversão) e somar $meio - i + 1$ (quantos restam em A), não 1. Ex. conferido: 1, 5, 4, 8, 10, 2, 6, 9, 12, 11, 3, 7 tem **22** inversões.

8.2 Subvetor contíguo de soma máxima (Teste 2, Exercício 1)

Dado $A[1..n]$ (inteiros, podem ser negativos), achar $i \leq j$ maximizando $A[i] + \dots + A[j]$ (subvetor **não vazio**). **Ideia:** o melhor subvetor de $A[ini..fim]$ está (i) inteiro na metade esquerda, (ii) inteiro na direita, ou (iii) **cruza o meio**, i.e., é $A[i..meio] \cup A[meio + 1..j]$. O caso (iii) se resolve em tempo linear: melhor sufixo da esquerda terminando em $meio$ + melhor prefixo da direita começando em $meio + 1$.

Subvetor de soma máxima por D&C (retorna soma, início, fim)

```

Função MAXSUB( $A, ini, fim$ )
  se  $ini = fim$  então
    retorna ( $A[ini]$ ,  $ini$ ,  $ini$ )                                     % não vazio: um elemento
   $meio := \lfloor (ini + fim)/2 \rfloor$ 
   $(s_E, i_E, j_E) := \text{MAXSUB}(A, ini, meio)$ 
   $(s_D, i_D, j_D) := \text{MAXSUB}(A, meio + 1, fim)$ 
   $(s_C, i_C, j_C) := \text{MAXCRUZADO}(A, ini, meio, fim)$ 
  retorna a tripla de maior soma entre  $(s_E, i_E, j_E)$ ,  $(s_D, i_D, j_D)$ ,  $(s_C, i_C, j_C)$ 

Função MAXCRUZADO( $A, ini, meio, fim$ )
   $soma := 0$ ;  $melhorE := -\infty$ 
  para  $i := meio$  decrecendo até  $ini$  faça
     $soma := soma + A[i]$ 
    se  $soma > melhorE$  então
       $melhorE := soma$ ;  $iMax := i$ 
   $soma := 0$ ;  $melhorD := -\infty$ 
  para  $j := meio + 1$  até  $fim$  faça
     $soma := soma + A[j]$ 
    se  $soma > melhorD$  então
       $melhorD := soma$ ;  $jMax := j$ 
  retorna ( $melhorE + melhorD$ ,  $iMax$ ,  $jMax$ )

```

- **Complexidade:** $T(n) = 2T(n/2) + \Theta(n) \Rightarrow \Theta(n \log n)$ (Cormen §4.1).
- **Todos negativos:** funciona, pois o caso base devolve um elemento e o cruzado usa pelo menos $A[meio]$ e $A[meio + 1]$; o máximo das três opções é o maior elemento. Ex. 2 do teste: $[-8, -3, -6, -2, -5, -4] \rightarrow -2$.
- Ex. 1 do teste (conferido): $[-2, 1, -3, 4, -1, 2, 1, -5, 4] \rightarrow$ soma 6, subvetor $[4, -1, 2, 1]$. *O enunciado indexa a partir de 0* (posições 3 a 6); com índices a partir de 1, posições 4 a 7.
- (Fora de D&C: o algoritmo de Kadane resolve em $\Theta(n)$ mantendo o melhor sufixo terminando em cada posição. Também existe D&C linear devolvendo (soma total, melhor prefixo, melhor sufixo, melhor) de cada metade: $T(n) = 2T(n/2) + O(1) = \Theta(n)$.)

8.3 Multiplicação de matrizes: Strassen (Teste 2, Exercício 2)

Produto por blocos ingênuo: 8 multiplicações de $n/2 \times n/2$, $T(n) = 8T(n/2) + \Theta(n^2) = \Theta(n^3)$. Strassen usa **7**: $T(n) = 7T(n/2) + \Theta(n^2) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2,807})$ (Teorema Mestre caso 1, pois $n^2 = O(n^{\log_2 7 - \epsilon})$).

Strassen(A, B, n) – n potência de 2; Soma/Subtrai são dadas

```

Função STRASSEN( $A, B, n$ )
  se  $n = 1$  então
    retorna  $[A[1, 1] \cdot B[1, 1]]$ 
  particione  $A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$ ,  $B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$  em blocos  $\frac{n}{2} \times \frac{n}{2}$ 
   $M_1 := \text{STRASSEN}(A_{11} + A_{22}, B_{11} + B_{22}, n/2)$ 
   $M_2 := \text{STRASSEN}(A_{21} + A_{22}, B_{11}, n/2)$ 
   $M_3 := \text{STRASSEN}(A_{11}, B_{12} - B_{22}, n/2)$ 
   $M_4 := \text{STRASSEN}(A_{22}, B_{21} - B_{11}, n/2)$ 
   $M_5 := \text{STRASSEN}(A_{11} + A_{12}, B_{22}, n/2)$ 
   $M_6 := \text{STRASSEN}(A_{21} - A_{11}, B_{11} + B_{12}, n/2)$ 
   $M_7 := \text{STRASSEN}(A_{12} - A_{22}, B_{21} + B_{22}, n/2)$ 
   $C_{11} := M_1 + M_4 - M_5 + M_7$ ;  $C_{12} := M_3 + M_5$ 
   $C_{21} := M_2 + M_4$ ;  $C_{22} := M_1 - M_2 + M_3 + M_6$ 

```

```

retorna  $\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$ 

```

São 18 somas/subtrações de matrizes $\frac{n}{2} \times \frac{n}{2}$ ($\Theta(n^2)$) por chamada. Na prática usa-se o método clássico abaixo de um certo n (constantes maiores, menos estabilidade numérica).

8.4 Outros exemplos clássicos de D&C (treino para a Q5)

Busca binária, potência rápida e máximo/mínimo

```

Função BUSCABIN( $V, ini, fim, x$ )                                %  $V$  ordenado;  $T(n) = T(n/2) + O(1) = O(\log n)$ 
  se  $ini > fim$  então
    retorna  $-1$ 
   $meio := \lfloor (ini + fim)/2 \rfloor$ 
  se  $V[meio] = x$  então
    retorna  $meio$ 
  senão se  $x < V[meio]$  então
    retorna BUSCABIN( $V, ini, meio - 1, x$ )
  senão
    retorna BUSCABIN( $V, meio + 1, fim, x$ )

Função POT( $a, n$ )                                              %  $a^n$  com  $O(\log n)$  multiplicações
  se  $n = 0$  então
    retorna  $1$ 
   $t := \text{POT}(a, \lfloor n/2 \rfloor)$ 
  se  $n$  par então
    retorna  $t \cdot t$ 
  senão
    retorna  $t \cdot t \cdot a$ 

Função MAXMIN( $V, ini, fim$ )                                    %  $\approx 3n/2$  comparações (contra  $2n - 2$  do ingênuo)
  se  $ini = fim$  então
    retorna ( $V[ini], V[ini]$ )
  se  $fim = ini + 1$  então
    retorna ( $\max, \min$ ) de  $V[ini], V[fim]$  com 1 comparação
   $meio := \lfloor (ini + fim)/2 \rfloor$ 
  ( $M_1, m_1$ ) := MAXMIN( $V, ini, meio$ ); ( $M_2, m_2$ ) := MAXMIN( $V, meio + 1, fim$ )
  retorna ( $\max\{M_1, M_2\}, \min\{m_1, m_2\}$ )

```

Elemento majoritário (aparece $> n/2$ vezes), $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$

```

Função MAJ( $V, ini, fim$ )                                       % retorna o majoritário de  $V[ini..fim]$  ou NENHUM
  se  $ini = fim$  então
    retorna  $V[ini]$ 
   $meio := \lfloor (ini + fim)/2 \rfloor$ 
   $a := \text{MAJ}(V, ini, meio)$ ;  $b := \text{MAJ}(V, meio + 1, fim)$ 
  para  $c \in \{a, b\}$ ,  $c \neq \text{NENHUM}$  faça
    se  $c$  ocorre mais de  $(fim - ini + 1)/2$  vezes em  $V[ini..fim]$  então
      retorna  $c$ 
  % contagem  $O(n)$ 

retorna NENHUM

```

Correção: se x é majoritário no todo, é majoritário em pelo menos uma das metades (senão ocorreria $\leq n/2$ vezes).

Mais ideias de D&C. *Karatsuba* (inteiros de n dígitos com 3 produtos de $n/2$: $\Theta(n^{\log_2 3}) \approx n^{1.585}$); *par de pontos mais próximos* no plano ($\Theta(n \log n)$: divide pela mediana de x , combina olhando a faixa central com no máximo 7 vizinhos por ponto); *pico em vetor unimodal* e *raiz de função monótona* (busca binária, $O(\log n)$); *contar ocorrências de x em vetor ordenado* (duas buscas binárias).

Erros frequentes em pseudocódigo de D&C

- Esquecer o caso base ou fazê-lo errado (ex.: subvetor *vazio* permitido quando o enunciado pede não *vazio*).
- Dividir em $[ini, meio - 1]$ e $[meio, fim]$ com $meio = \lfloor (ini + fim)/2 \rfloor$: com 2 elementos gera recursão infinita. Use $[ini, meio]$ e $[meio + 1, fim]$.
- Combinação que olha só um elemento de cada lado (o subvetor cruzado precisa do *melhor sufixo* e do *melhor prefixo*).
- Não devolver a informação extra necessária (índices; vetor ordenado nas inversões).
- Não dizer a complexidade (o enunciado sempre pede!).

Pontos-chave

D&C = dividir (meio), conquistar (2 chamadas), combinar (o que cruza o meio). $2T(n/2) + \Theta(n) = \Theta(n \log n)$; $T(n/2) + O(1) = \Theta(\log n)$; $7T(n/2) + \Theta(n^2) = \Theta(n^{2,807})$; $T(n/5) + T(3n/4) + \Theta(n) = \Theta(n)$.

9 Complementos

9.1 Tabelas hash (pesquisa)

Complemento – pesquisa proposta em 02/09, entrega 19/09 (Cormen, cap. 11; Szwarcfiter, cap. 10)

Motivação: implementar um *dicionário* (inserir, buscar, remover por chave) com acesso “direto”. Endereçamento direto (vetor indexado pela chave) é $O(1)$ mas exige memória do tamanho do universo de chaves (infinito ou enorme). Uma **função hash** $h : U \rightarrow \{0, \dots, m - 1\}$ transforma a chave num endereço de uma tabela de m posições.

Funções clássicas: *divisão* $h(k) = k \bmod m$ (m primo longe de potências de 2); *multiplicação* $h(k) = \lfloor m(kA \bmod 1) \rfloor$ ($A \approx (\sqrt{5} - 1)/2$); *dobra/meio do quadrado*; para strings, polinomial $h(s) = \sum s_i b^i \bmod m$; *hashing universal* (escolher h aleatoriamente de uma família, p.ex. $((ak + b) \bmod p) \bmod m$), que garante bom caso esperado para qualquer entrada.

Colisões ($h(k_1) = h(k_2)$, $k_1 \neq k_2$) são inevitáveis (princípio da casa dos pombos: $|U| > m$). Tratamento:

- **Encadeamento exterior:** cada posição aponta para uma lista das chaves com aquele endereço.
- **Encadeamento interior:** as listas usam posições da própria tabela (uma zona de colisões, ou ponteiros entre posições livres).
- **Endereçamento aberto:** tudo fica na tabela; em colisão, sonda-se uma sequência de posições: *linear* $h(k) + i$ (agrupamento primário), *quadrática* $h(k) + c_1 i + c_2 i^2$, *duplo hashing* $h_1(k) + i h_2(k)$. Remoção exige marca “removido”.

Complexidade: com fator de carga $\alpha = n/m$ e hash uniforme, busca em $\Theta(1 + \alpha)$ esperado com encadeamento, e $\leq \frac{1}{1-\alpha}$ sondagens esperadas (busca sem sucesso) no endereçamento aberto; mantendo $\alpha = O(1)$ (redimensionando a tabela, custo amortizado $O(1)$), as operações são $O(1)$ esperado. **Pior caso:** $\Theta(n)$ (todas as chaves no mesmo endereço). Hash não mantém ordem (sem mínimo, sucessor, intervalos — aí usa-se AVL/rubro-negra).

9.2 Programação dinâmica

Complemento – slides de PD (subsequência comum máxima); não previsto no formato da AV1

Na decomposição recursiva, **subproblemas idênticos** podem aparecer muitas vezes, gerando tempo exponencial mesmo com poucos subproblemas distintos (ex.: Fibonacci recursivo). **PD**: guardar a solução de cada subproblema numa tabela e resolvê-los de baixo para cima (*bottom-up*) ou com memorização. D&C é bom quando os subproblemas são disjuntos; PD quando eles se sobrepõem.

Subsequência comum máxima (LCS). $X = x_1..x_n$, $Y = y_1..y_m$; $C(i, j)$ = tamanho da LCS de X_i e Y_j :

$$C(i, j) = \begin{cases} 0 & i = 0 \text{ ou } j = 0 \\ 1 + C(i - 1, j - 1) & x_i = y_j \\ \max\{C(i - 1, j), C(i, j - 1)\} & x_i \neq y_j \end{cases}$$

Preenche-se a tabela linha a linha: $O(nm)$ tempo e espaço; resposta $C(n, m)$. Ex.: $X = 5, 2, 1, 3, 9, 10, 14$, $Y = 2, 3, 1, 10$: LCS de tamanho 3 (2, 3, 10 ou 2, 1, 10). Para *recuperar* a subsequência, volte de (n, m) : se $x_i = y_j$, x_i faz parte e vá a $(i - 1, j - 1)$; senão vá para a vizinha de maior valor.

10 Banco de questões com respostas

10.1 Testes aplicados

Teste 1 (heaps, Union-Find, AVL).

1. Aeroporto: filas de prioridade por setor, fusões muito frequentes, alterações raras, tempo *previsível por operação*. Qual heap? $\Rightarrow$ **binomial** (§3.4).
2. Central médica: $2 \cdot 10^6$ chamadas de 16 bytes, limite de 40 MB, ponteiros de 8 bytes, sem fusão, poucas alterações. $\Rightarrow$ **heap binário** (§3.4, conta de memória).
3. Supercomputador: $5 \cdot 10^6$ inserções/remoções e $8 \cdot 10^7$ aumentos de prioridade, custo total importa. $\Rightarrow$ **Fibonacci** (§3.4).
4. Plataforma com 10^7 usuários: 10^5 uniões e 10^8 consultas; árvore vs. lista; estratégia de união. $\Rightarrow$ Union-Find, lista com ponteiro ao representante (FIND $O(1)$), união “menor na maior” (§4.4).
5. AVL: árvore 50(30(20(10, 25), 40), 70(60, 80)), inserir 5, 15, 35, 45, identificar desregulados e rotações; ABB vs. AVL $\Rightarrow$ Exemplo 5.5.

Teste 2 (ordenação e D&C, 2h, 3 questões).

1. Subvetor contíguo de soma máxima por D&C, com posições e complexidade $\Rightarrow$ §8.
2. Strassen: algoritmo recursivo com as 7 multiplicações $\Rightarrow$ §8.
3. Escolher Counting/Radix/Bucket/Merge/Quick para 5 cenários $\Rightarrow$ §6.4.

10.2 Lista – heaps binários, binomiais e de Fibonacci

Obs.: a numeração segue a da lista, que **pula o exercício 9** (vai do 8 ao 10).

1. *Verificar se são heaps.* (a) 33 32 28 31 26 29 25 30 27: **não** — posição 6 (29) tem pai na posição 3 (28) e $29 > 28$. (b) 33 32 28 31 29 26 25 30 27: **sim** (todas as 8 relações pai-filho valem).
2. *Troca com $i < j$, $s_i < s_j$ gera heap?* Falso: [10, 9, 1, 8, 7, 0, 0, 6], troca das posições 3 e 4 (§2.7).
3. *Idem com $s_i > s_j$.* Falso: [10, 9], troca das posições 1 e 2.

4. Número de permutações que são min-heap. $H(n) = \binom{n-1}{L} H(L) H(n-1-L)$; valores 1, 1, 2, 3, 8, 20, 80, 210, ... (§2.7).
5. Construção linear de 18 25 41 34 14 10 52 50 48. [52, 50, 41, 48, 14, 10, 18, 34, 25] (traço em §2).
6. No máximo $\lceil n/2^j \rceil$ nós de altura j . Lema 2.3.
7. Construir heap binomial em $O(n)$. n inserções sucessivas: o custo total é o do contador binário, $\sum_k n/2^k \leq 2n$ links (§3.1). Alternativa: formar pares $B_0 + B_0 \rightarrow B_1$, depois pares de B_1 , etc., como uma soma binária de n uns.
8. Alturas para o pior caso da inserção num heap com k árvores. Quando o novo B_0 provoca “vai um” em todas as posições: o heap contém $B_0, B_1, \dots, B_{k-1}$ ($n = 2^k - 1$, todos os bits 1), isto é, alturas $0, 1, \dots, k-1$ em arestas ($1, \dots, k$ em níveis, convenção do Szwarcfiter). São feitos k links e o resultado é uma única B_k .
10. Remoção do máximo no pior caso. A remoção custa percorrer as k raízes, gerar os h filhos da raiz removida e fundir H' (as outras $k-1$ árvores) com $H'' = \{B_0, \dots, B_{h-1}\}$. O máximo de links ocorre quando H' também contém $B_0, \dots, B_{h-1}$ (cada posição gera “vai um”) e h é o maior possível: $H = \{B_0, B_1, \dots, B_{k-1}\}$ **com o máximo na raiz de B_{k-1}** , logo $h = k-1$ (em arestas).
11. $k \geq \min\{k_1, k_2\}$? **Falso:** H_1 com $3 = 11_2$ nós ($k_1 = 2$), H_2 com $5 = 101_2$ ($k_2 = 2$); a fusão tem $8 = 1000_2$ nós, $k = 1 < 2$.
12. $k \leq k_1 + k_2$? **Verdadeiro:** a união inicial tem $k_1 + k_2$ árvores e cada link transforma duas árvores em uma; logo $k = k_1 + k_2 - \#links \leq k_1 + k_2$.
13. Altura mínima e máxima de heap de Fibonacci em função do posto k . Hipótese do exercício (Szwarcfiter §9.5): árvores de Fibonacci de posto k , formadas por subordinações de árvores de mesmo posto e cortes controlados por marcas em nós *não raiz*. **Mínima:** pelo lema $\text{grau}(y_i) \geq i-2$, a menor altura $A(k)$ satisfaz $A(k) = 1 + A(k-2)$, $A(0) = 0$, $A(1) = 1$, logo $A(k) = \lceil k/2 \rceil$ (atingida pelas árvores de Fibonacci mínimas). **Máxima:** k , atingida pela própria B_k (por indução: subordinar duas árvores de posto h e altura $\leq h$ dá posto $h+1$ e altura $\leq h+1$; cortes não aumentam a altura nem mudam o posto da raiz). Alturas em arestas. (Observação fora do escopo do exercício: numa implementação geral, em que a raiz pode perder filhos livremente, a altura não é limitada por uma função do posto — o que é sempre $O(\log n)$ é o posto/grau.)
- 14–15. Algoritmos dos heaps binomiais e de Fibonacci. §3.1 e §3.3.
16. Heap binário sem recursão.

Subir e descer iterativos (ex. 16; Szwarcfiter ex. 6.7–6.8)

```

Procedimento SUBIR( $i$ )
  enquanto  $i > 1$  e  $T[i].chave > T[\lfloor i/2 \rfloor].chave$  faça
     $T[i] \Leftrightarrow T[\lfloor i/2 \rfloor]$ ;  $i := \lfloor i/2 \rfloor$ 
Procedimento DESCER( $i, n$ )
  enquanto  $2i \leq n$  faça
     $j := 2i$ 
    se  $j < n$  e  $T[j+1].chave > T[j].chave$  então
       $j := j+1$ 
    se  $T[i].chave < T[j].chave$  então
       $T[i] \Leftrightarrow T[j]$ ;  $i := j$ 
  senão
    pare
  
```

Inserção, remoção, alteração e construção usam exatamente os mesmos esquemas da §2 com essas versões.

10.3 Lista – Union-Find e AVL

1. União por rank sem compressão: altura $\leq \lfloor \log n \rfloor$ e sequência que atinge $\Theta(\log n) \Rightarrow$ Teo. 4.1 e exemplos da §4.
2. Só compressão: sequência que gera árvore alta e custo $O(m \log n) \Rightarrow$ §4.
3. Remoção em AVL com rotação em todos os ancestrais $\Rightarrow$ árvore de Fibonacci T_7 , remover a folha 33 (§5).
4. Fusão de duas AVL $\Rightarrow$ junção pela espinha direita $O(\log n)$, ou intercalação dos percursos em ordem $O(n + m)$ (§5).
5. AVL como conjuntos disjuntos: FIND $O(\log n)$, UNION $O(k \log n) \Rightarrow$ §5.

10.4 Monitoria de revisão: respostas curtas

- **UF-1.** Objetivo: manter partição em conjuntos disjuntos com consultas “mesmo conjunto?” e uniões. MAKE-SET cria $\{x\}$ ($\text{pai}[x] = x$); FIND sobe até a raiz; UNION liga uma raiz na outra. Cada elemento armazena o **pai** (e a raiz, o rank). União por rank: liga o de menor rank sob o de maior; altura $\leq \log n$.
- **AVL-1.** Balanceada: altura $O(\log n)$ em toda subárvore. AVL: $|h_D - h_E| \leq 1$ em todo nó. Toda AVL é balanceada (Teo. 5.3), nem toda balanceada é AVL. Fator de balanço $\text{bal}(v) = h_D(v) - h_E(v)$.
- **AVL-2** (inserção à esquerda de p): $\text{bal} = 1 \rightarrow 0$, altura não muda, para; $\text{bal} = 0 \rightarrow -1$, altura aumenta, continua; $\text{bal} = -1 \rightarrow -2$, rotação e para.
- **AVL-3.** $\text{bal}(p) = -2$, $\text{bal}(u) = -1$: simples à direita. $\text{bal}(p) = -2$, $\text{bal}(u) = +1$: dupla à direita (esquerda em u , direita em p). Diferença: no primeiro o excesso está “em linha” (esquerda-esquerda) e uma rotação resolve; no segundo está em “zigue-zague” (esquerda-direita) e é preciso primeiro alinhar.
- **Heap-1.** Definição $s_i \leq s_{\lfloor i/2 \rfloor}$; seleção $O(1)$ porque o máximo está na posição 1 (Prop. 2.6); custos: $O(1)$, $O(\log n)$, $O(\log n)$, $O(\log n)$, $O(n)$.
- **Binomial (Q6–Q8, Q11).** B_k : duas B_{k-1} ligadas; 2^k nós; grau da raiz k ; o heap é a lista de raízes com ordens distintas (bits de n). Link de duas B_2 de raízes 10 e 15 (min-heap): raiz 10, a de 15 vira sua filha; resultado B_3 com 8 nós e grau 3. Mínimo: percorre as $O(\log n)$ raízes (ou ponteiro). União $O(\log n)$: só percorre as listas de raízes (não os n nós), fazendo links como numa soma binária.
- **Fibonacci (Q9, Q10, Q12).** Diferença estrutural: permite várias árvores de mesma ordem (preguiçoso), listas circulares duplas, marcas. $O(1)^*$: inserção, união, aumento de prioridade (*decrease-key* no min-heap), seleção. Remoção do extremo: $O(\log n)^*$. Aumentar prioridade pode violar a ordem com o pai; corta-se x e o leva à lista de raízes; cortes em cascata sobem pelos pais marcados; custo $O(1)^*$ porque cada corte em cascata é pago pela operação que marcou o nó (potencial $t + 2m$).

Checklist final para a AV1

1. **AVL:** inserir/remover desenhando a árvore e os balanços a cada passo; rotacionar no desregulado *mais profundo*; saber os 4 casos e o pseudocódigo (papel de h e de pt).
2. **Union-Find:** tabela de implementações e cenários (listar membros, rollback, offline, Kruskal); provar altura $\leq \log n$ com rank.
3. **Heap:** provas por caminho/transitividade, índices e indução; construção $O(n)$; contraexemplos pequenos.
4. **Escolha:** identificar a operação dominante e o tipo da chave; justificar com complexidade,

memória, previsibilidade (pior caso vs. amortizado), estabilidade.

5. **D&C**: caso base, divisão, duas chamadas, combinação do “cruzamento”, recorrência resolvida.